

MANCHESTER METROPOLITAN UNIVERSITY



FINAL YEAR DISSERTATION

An Inter-procedural Framework for Memory Leak Detection in C using Large Language Models

Author:
Jacob Wallis
22513465

Supervisor:
Matthew Shardlow

Degree:
Computer Science

Department:
Computing and Mathematics

Faculty:
Science and Engineering

May 2026

Acknowledgements

I would like to express my gratitude to my supervisor, Matthew Shardlow, for their guidance and support over the past year. I also thank Claire, Dave and Rowan Wallis for their encouragement and care. Finally, thanks to David Thomson for inspiring my career and all my projects.

Abstract

This report details a framework for interprocedural memory leak detection in the C programming language using Large Language Models (LLMs). There are a multitude of traditional approaches to memory leak detection. While traditional static analysis enables early identification of errors, it often exhibits structural rigidity and requires significant manual configuration. Dynamic Analysis identifies runtime vulnerabilities and performance issues by evaluating the software during execution. However, dynamic analysis is inherently limited by specific runtime conditions or input sets and fails to uncover vulnerabilities contained within unexecuted control flow paths. Memory leaks are notoriously insidious, often remaining latent until causing cumulative performance degradation in production. Memory leaks are traditionally exposed through syntactic matching, exhaustive value and control flow analysis, or tracking pointer reachability. This report presents Leak Logic Mapper, an interprocedural framework that instead utilises semantic reasoning and context adjacent abstraction to resolve heap operations. Recent advances in Natural Language Processing have allowed for a reconceptualisation of software analysis methodologies, utilising LLMs for peer reviews, error handling and bug ramification. Traditional static analysis methods struggle to interpret semantic intent, whereas LLMs demonstrate a high aptitude for interpreting underlying logic, being able to execute instructions with non-standard context and patterns. However, LLMs suffer from context decay when tasked with processing extensive token volumes, inherent with large code bases. Leak Logic Mapper minimises context decay by performing compositional bottom-up topological analysis on a C codebase, and abstracting function logic into validated hierarchical ‘memory profiles’. Evaluation of the framework demonstrates an ability to resolve semantic intent in implementations derived strictly from raw C code; further analysis successfully identifies nuanced runtime errors resulting in intricate conditional leaks. The framework constrains LLM analysis into highly architected JSON responses, to validate, pinpoint and profile leaks within source code. Evaluation of the framework against industry-standard tools, including Valgrind, AddressSanitizer, Cppcheck and Clang demonstrates that Leak Logic Mapper successfully identifies complex, interprocedural vulnerabilities that elude traditional static and dynamic analysis methods.

Table of Contents

Declaration.....	i
Acknowledgements.....	ii
Abstract.....	iii
List of Figures.....	vi
List of Tables.....	vii
1. Introduction.....	1
1.1 Project Background.....	1
1.2 Aim and Objectives.....	2
1.3 Dissertation Structure.....	3
2. Literature Review.....	4
3. Design.....	7
3.1 Core Requirements.....	7
3.2 Prompting.....	18
3.3 Deterministic Boundary Extraction (AST).....	20
3.4 Tag Generation.....	21
3.5 Concurrent Execution.....	22
3.6 Command-Line Interface and Diagnostic Reporting.....	22
3.7 High Level Design Diagram.....	24
3.8 Architectural Modularity: Cloud API and Local Backends.....	25
3.9 System Scope (Minimum Viable Product Constraints).....	25
4. Implementation.....	28
4.1 Project Setup.....	28
4.2 Testing.....	44
5. Evaluation.....	59
5.1 Core Criteria:.....	59
5.2 Evaluating LLM Comprehension.....	60
5.3 Static Analysis Comparison.....	61
5.4 The Realloc Anti-Pattern.....	68
5.5 Opaque Boundaries and Unpredictable Variance.....	70
5.6 The Model Capability Problem.....	73
5.7 Evaluation Metrics.....	73
5.8 Evaluation Synthesis.....	76
6. Conclusion.....	77

6.1 Completed Work.....	77
6.2 Limitations	78
6.3 Future Work.....	79
References.....	84
Appendices.....	87

List of Figures

Figure 3-1: Source Tag Examples.....	12
Figure 3-2: Sink Tag Examples	12
Figure 3-3: PassThrough Tag Examples.....	14
Figure 3-4: Reference Loss Tag Examples.....	15
Figure 3-5: InternalLeak Tag Examples	15
Figure 3-6: MemoryNeutral Tag Example	16
Figure 3-7: Nested sources and sinks (Flowchart).....	16
Figure 3-8: Nested sources and sinks (code)	17
Figure 3-9: Exemplar terminal output for an isolated memory leak in a single function C program.	23
Figure 3-10: High Level Design Diagram	24
Figure 4-1: .env example file.	29
Figure 4-2: Directory structure	29
Figure 4-3: tree-sitter-c queries.....	30
Figure 4-4: Return extraction from the AST and the LLM.	32
Figure 4-5: JSON Schema Sources Object	34
Figure 4-6: Schema Class	35
Figure 4-7: System Instruction - Child Function Tag Rules.....	35
Figure 4-8: ALLOCA Validation Check	41
Figure 4-9: Local Scope Reference Loss	43
Figure 4-10: Missing Free Test Case	48
Figure 4-11: Conditional Free Test Case	52
Figure 4-12: Alloca Test Case	55
Figure 4-13: Conditional Leak Test Case	58
Figure 5-1: Cppcheck Intraprocedural Memory Leak	62
Figure 5-2: Interprocedural Memory Leak	62
Figure 5-3: Cppcheck configuration file.....	63
Figure 5-4: Cppcheck Leak Acknowledgement	63
Figure 5-5: Cppcheck False Negative.....	64
Figure 5-6: Clang Static Analyzer, Memory Leak.....	65
Figure 5-7: Clang Static Analyzer, Test Case 13.....	66
Figure 5-8: Nested Heap Leak	67
Figure 5-9: Leak Logic Mapper Nested Heap Leak	68
Figure 5-10: Clang Static Analyzer Nested Heap Leak.....	68
Figure 5-11: Realloc Leak	69
Figure 5-12: Unsupported Callback Function Test Case.....	71
Figure 5-13: Mispopulated Source Callback Function Test Case.....	72

List of Tables

Table 3-1 Tagging Reference.....	10
Table 4-1 Sources Tag Routes.	40
Table 4-2: Sinks Tag Routes.....	41
Table 4-3: PassThrough Tag Routes.....	42
Table 4-4: ReferenceLoss Tag Routes	42
Table 4-5: Test Case Overview.....	45

Abbreviations

Abbreviation	Description
ASan	Address Sanitizer
AST	Abstract Syntax Tree
CFG	Control Flow Graph
CLI	Command Line Interface
CoT	Chain-of-Thought
DAG	Directed Acyclic Graph
GCC	GNU Compiler Collection
JSON	JavaScript Object Notation
LLM	Large Language Model
MVP	Minimum Viable Product
SAST	Static Application Security Testing
VFG	Value-Flow Graph

1. Introduction

1.1 Project Background

The C programming language grants developers manual responsibility over memory management. While providing significant performance advantages, this explicit control introduces inherent security risks. Memory leaks are particularly problematic; rather than triggering immediate crashes, they cause long-term system degradation. Given that modern digital infrastructure remains ubiquitously dependent on C, such vulnerabilities are not confined to contemporary implementations; indeed, latent memory leaks continue to be uncovered within mature, legacy software. The application of advanced diagnostic tools, exemplified by those evaluated within this paper, frequently exposes deeply nested leaks within foundational systems, from the Linux kernel to core command-line utilities.

Current methodologies for identifying memory leaks predominantly fall into two imperfect paradigms. Dynamic analysis tools offer high precision but are inherently limited by runtime code coverage. Conversely, traditional static analysis tools evaluate uncompiled code but frequently suffer from context blindness, struggling to determine whether custom wrapper functions safely handle or mishandle pointers.

Traditional static analysers rely on syntactic pattern matching or rigid graph representations. While mathematically sound, these methods are incapable of interpreting developer intent or non-standard coding architectures. Furthermore, purely mathematical approaches fail to leverage the operational logic embedded within self-documenting code. This creates a "semantic gap," wherein an analyser might erroneously flag a safely managed pointer as a leak simply because it cannot recognise the logic of a developer's custom deallocation routine.

Initial attempts to apply Artificial Intelligence tooling to this domain explored using classification models to calculate a risk probability for memory leaks within isolated functions (see Appendix F: Feasibility Study). Under this methodology, high-risk functions would subsequently be subjected to targeted dynamic testing. However, this approach was discarded upon the realisation that it could not effectively track pointer lifecycles across a wider, inter-procedural program state.

The emergence of LLMs presented a highly capable alternative. Unlike simplistic classification models, LLMs excel at semantic reasoning and are uniquely positioned to bridge the

aforementioned ‘semantic gap’ by interpreting and relaying developer intent. Consequently, the project shifted focus to explore how LLMs could be leveraged to track pointer states systematically across function boundaries.

The project proposed a semantic tagging system for functional abstraction. By evaluating functions in isolation and assigning them formal tags based on their explicit memory contracts, the framework abstracts complex internal logic into concise ‘memory profiles’. These labelled profiles act as contextual binding blocks, enabling a methodical, bottom-up approach to interprocedural analysis that preserves wider program state without overwhelming the underlying model.

1.2 Aim and Objectives

The primary aim of this project is to architect an inter-procedural static analysis framework—Leak Logic Mapper—that leverages Large Language Models to detect memory leak vulnerabilities in C. The framework aims to overcome the traditional limitations of LLM token constraints by safely abstracting heap operation logic, thereby allowing the model to track complex pointer lifecycles across a wider program state without suffering from context decay.

The objectives of this project are as follows:

- **Minimise Context Decay:** Implement a topological, bottom-up pipeline capable of analysing isolated code segments individually. This will reduce token overhead and prevent context decay that inevitably occurs when LLMs are forced to process massive, multi-scope codebases simultaneously.
- **Abstract Heap Operations:** Design a methodology to translate the probabilistic semantic reasoning of an LLM into a structured, reliable data format. This must abstract a function’s core heap logic into a strict memory profile suitable for downstream analysis.
- **Resolve Inter-procedural Analysis:** Engineer a mechanism to preserve and propagate the transitional state of pointers across nested architectures, ensuring accurate tracking as state representations shift between caller and callee scopes.

- **Evaluate System Efficacy:** Benchmark the developed framework against traditional, industry-standard static and dynamic analysis tools to assess its accuracy, reliability and capability to identify complex memory leak vulnerabilities.

1.3 Dissertation Structure

The remainder of this dissertation is structured as follows: Chapter 2 reviews the existing literature surrounding static analysis methodologies and memory leak detection. Chapter 3 outlines the architectural design and core requirements of the Leak Logic Mapper framework. Chapter 4 details the practical implementation and testing of the system. Chapter 5 evaluates the framework's performance against industry-standard analysis tools and discusses nuanced observations made during development and testing. Finally, Chapter 6 provides conclusions, acknowledges system limitations, and outlines areas for future work.

2. Literature Review

Alanyan *et al.* (2022), survey a comprehensive overview of static analysis methods for memory leak detection and evaluate their performance upon the Juliet Test Suite (Boland and Black, 2012, pp. 88-90). The methods utilise static analysis and typically employ a two-stage architecture for leak detection. First, source code is converted into graph representations to identify candidate paths where memory allocation occurs but a deallocation call cannot be reached. Second, candidates are subjected to symbolic execution to prune false positives, by checking if a path is feasible.

The choice of graph representation dictates the precision of analysis. A Control Flow Graph (CFG) is a directed graph where nodes represent basic code blocks and edges represent control flow statements (e.g., if-else, loops). While a CFG can determine whether a `free()` call can be reached along an execution path, it will not capture data dependencies and will fail to distinguish which specific variable is freed. Conversely, a Value-Flow Graph (VFG) models the program as a network of memory constraints. In a VFG, nodes represent expressions that define or use values (such as `malloc()`), and edges represent the transfer of values via assignment or dereference (Sui and Xue, 2016). While VFGs allow for a precise model that can track pointer aliasing, they incur a significant construction cost. This process is often bottlenecked by pointer analysis algorithms, such as Andersen's Points-To Analysis (Andersen, 1994), which has a worst case time complexity of $O(n^3)$ (Aldrich, 2013).

Aslanyan *et al.* (2024) propose a hybrid architecture termed 'Memory Leak Hunter'. Their approach was built on the observation that where symbolic execution offers high precision, through the mathematical proof of a bug, it is a computationally expensive process, not well suited to analyse large-scale projects. Addressing the scalability bottleneck, symbolic execution is directed to a select number of execution paths identified as vulnerable. The first stage of the solution (the hunter) constructs a Procedure Graph, combining instructions, global variables and function arguments with data and control flows, to identify execution paths where the count of memory allocations exceeds deallocations. This mechanism, implemented as an 'Alloc Free Imbalance Checker', isolates suspicious paths and is detailed in the follow up work by Hovhannisyan *et al.* (2025). This static approach minimises false negatives though suffers from a high false positive rate, being unable to evaluate complex runtime constraints. The

second stage filters results by passing suspicious execution paths to a directed symbolic execution verifier to validate their feasibility.

Following the recognition of the structural constraints inherent in mathematical solvers, foundational security research began exploring the broader potential of LLMs to bridge the semantic gap. In a comprehensive evaluation presented at the USENIX Security Symposium, Fang *et al.* (2024) investigated the efficacy of LLMs in software reverse engineering and vulnerability detection. A key finding of their work highlights the historical dichotomy between deterministic tools, which rely on extracting structural relationships into an Abstract Syntax Tree (AST) for rigid pattern-matching, and modern LLMs, which excel at abstract semantic reasoning but struggle with deterministic accuracy. Fang *et al.* note that while LLMs demonstrate significant potential in analysing complex code logic, their standalone application is often hindered by a lack of structured, deterministic grounding. This foundational critique encapsulates the "semantic-structural gap" in modern code analysis. Demonstrating that the future of vulnerability detection lies in combining the rigid structural boundaries of traditional ASTs with the nuanced semantic reasoning of LLMs.

In a systematic head-to-head evaluation, Gnieciak and Szandała (2025) found that LLMs significantly outperformed industry-standard Static Application Security Testing (SAST) tools, such as SonarQube (SonarSource, 2026) and CodeQL (GitHub, 2026), in overall recall, achieving mean F1 scores of up to 0.797 compared to SonarQube's 0.260 and CodeQL's 0.386. The authors attribute this to the LLM's capacity to reason across broader inter-procedural contexts rather than relying on predefined signature rules. However, the study explicitly cautions against the standalone use of LLMs in safety-critical audits, noting that models frequently mislocate vulnerabilities at line-level due to "tokenisation artefacts" and produce high false-positive ratios.

Furthermore, recent empirical studies caution against naive LLM implementation strategies. In a comprehensive benchmark of state-of-the-art models against real-world C/C++ codebases, Ahmed *et al.* (2025) demonstrated that LLMs perform exceptionally poorly when tasked with standard function-level binary classification. A deeper examination of their methodology reveals a fundamental architectural vulnerability inherent to current "agentic" pipelines: to resolve missing information, the researchers utilised a prompting strategy that required the LLM itself to identify and request the additional external symbols (such as macros, globals, or structs) necessary to detect a vulnerability. This introduces a paradoxical dependency, relying

on the model's non-deterministic reasoning to identify its own contextual blind spots. If an LLM lacks the deep structural awareness required to recognise the relevance of missing context, it will inherently fail to request it. Consequently, this establishes a critical design imperative: to guarantee analytical rigor, a framework cannot rely on unstructured zero-shot prompts or agentic context-gathering. Instead, it must utilise deterministic, programmatic pre-processing, such as comprehensive AST extraction, to systematically bind and inject all relevant inter-procedural states into the prompt before any semantic evaluation occurs.

Finally, the limitations of current benchmarking datasets explicitly dictate the required scope for next-generation tools. As Ahmed *et al.* concede, their evaluation dataset inherently "does not extend to repository-wide contexts, such as transitive effects of non-local interactions beyond immediate function calls." This admission highlights a critical frontier in LLM centric security audits. It establishes that for an LLM-based framework to be practically viable in enterprise environments, it cannot merely analyse isolated functions. Instead, it must be architected to perform deep inter-procedural analysis, tracing memory states up a call graph. Addressing this specific architectural gap forms the foundational motivation for the methodology proposed in the following chapter.

The timeliness and necessity of this architectural approach is further evidenced by highly concurrent research in the field. After the initial development of Leak Logic Mapper was underway, Huang *et al.* (2026) published a neuro-symbolic framework for memory leak detection that independently mirrors several of this project's core objectives. Specifically, their work validates the necessity of utilising LLMs to generate structured 'summaries' of memory ownership to augment deterministic static analysers. The parallel emergence of such methodologies in the latest 2026 literature serves as validation that the architecture proposed in this dissertation represents the current state-of-the-art trajectory for C/C++ vulnerability analysis.

3. Design

This chapter proposes the architectural design for Leak Logic Mapper, a purpose-built tool for static memory analysis.

3.1 Core Requirements

The primary objective of Leak Logic Mapper is to utilise a LLM to formally verify and categorise memory profiles within uncompiled C code. To mitigate the inherent risk of hallucinations and ensure analytical reliability, Leak Logic Mapper is built against a strict set of operational requirements.

3.1.1 Formal Memory Profiling

The system shall construct a definitive memory profile for each evaluated function. If a function executes a heap operation that alters the broader program state, for instance, accepting a pointer and freeing it (a sink) or allocating memory to a pointer which exits the functions scope (a source), it is formally tagged as such. A semantic natural language summary detailing this function's operational lifecycle is generated. Tags are paired with the semantic summary to form a memory profile; profiles are injected with the prompt as part of a context payload during the subsequent analysis of its caller. This is termed a child to parent dependency, and forms the backbone of interprocedural analysis. Memory profiles serve to semantically abstract each function. It would dilute the model's attention and be highly susceptible to context decay to feed an LLM an entire codebase simultaneously. This abstraction allows the system to analyse functions individually, while preserving their structural side effects.

3.1.2 Deterministic Abstract Syntax Tree parsing & Call Graph Generation

The system natively parses C code using the Python library tree-sitter-c (Python Software Foundation, 2024; Tree-sitter, 2025) to extract deterministic boundaries from an AST including function signatures, parameters, and child function calls. This structural data is leveraged at run time to dynamically establish an implicit Directed Acyclic Graph (DAG) representing an exact function call hierarchy. This topological structure enables bottom-up, parallel analysis. Execution begins with absolute leaf nodes: functions that make no calls to other user-defined

functions within the program. Following the successful evaluation of all child functions, a parent function can be unblocked and dynamically queued for processing.

3.1.3 Constrained Semantic Extraction

The LLM is delegated to interpret semantic behaviour and declare what happens to memory both inside of a function and at that function's boundaries. Information is extracted as objects that map each pointer's transitional state. To ensure programmatic stability, the objects the LLM outputs are constrained to a predefined JSON schema. The system enforces this structured JSON output at the API level via OpenAI's native JSON mode, using the parameter: *response_format={"type": "json_object"}* (OpenAI, 2026).

3.1.4 Interprocedural Tagging

Once retrieved, objects mapping transitional pointer states are evaluated against a deterministic rule-based system. A Python orchestrator applies strict algorithmic validation to cross reference LLM responses against extracted boundaries from the AST and generate formal tags. All memory profiles will be attributed with at least one tag, categorised into one of two groupings:

- **Sources, Sinks, Passthroughs and ReferenceLosses:** These tags represent boundary-crossing operations. They directly dictate that a memory reference has crossed into or out of a child function's local scope, functions with these tags have the potential to alter the parent function's state.
- **Leaks and Neutral Operations:** These tags represent contracts isolated entirely within the child function's evaluation scope. They indicate that where heap operations may have occurred, they were either resolved or circumscribed within the function itself preventing interprocedural side effects from being passed up the call graph.

A function may be assigned any number of Source, Sink, Passthrough, ReferenceLoss or Leak tags, though in a well-architected codebase, in which functions adhere to the single responsibility principle, the proliferation of tags on any function is naturally minimised. A function is said to be memory neutral if it was not designated at least one of these tags.

When analysing the target function, the LLM receives exactly the memory profiles that have been generated for all the child functions it makes calls to. The LLM is entirely blinded to the child's internal code. While this modular approach successfully minimises context decay, it dictates that inter-procedural analysis depends upon fidelity of the profiling system; if a child's

memory profile is inaccurate, it will not accurately encapsulate side effects and the parent's subsequent analysis may be subject to failure.

Category	Tag Format	Definition
Sources	AllocSource<var_type>::Ret	A new pointer is allocated and unconditionally returned.
	CondAllocSource<var_type>::Ret	A new pointer is conditionally allocated and returned
	AllocSource<var_type>::Arg{dstIdx}	A new pointer is allocated and unconditionally assigned to an out-parameter
	CondAllocSource<var_type>::Arg{dstIdx}	A new pointer is conditionally allocated and assigned to an out-parameter argument.
Sinks	FreeSink<var_type>::Arg{srcIdx}	A pointer argument is unconditionally freed.
	CondFreeSink<var_type>::Arg{srcIdx}	A pointer argument is conditionally freed.
Passthroughs	PassThrough<var_type>::Arg{srcIdx}::ToRet	A pointer enters via an argument and unconditionally exits via the return statement.
	CondPassThrough<var_type>::Arg{srcIdx}::ToRet	A pointer enters via an argument and conditionally exits via the return statement.
	PassThrough<var_type>::Arg{srcIdx}ToArg{dstIdx}	A pointer enters via the srcIdx argument and is unconditionally written into the memory address provided by the dstIdx argument.
	CondPassThrough<var_type>::Arg{srcIdx}ToArg{dstIdx}	A pointer enters via the srcIdx argument and is

		conditionally written into the memory address provided by the dstIdx argument.
Reference Losses	ReferenceLoss<var_type>::Arg{srcIdx}	A function accepts a double pointer and destructively overwrites its dereferenced value.
	CondReferenceLoss<var_type>::Arg{srcIdx}	A function accepts a double pointer and destructively overwrites its dereferenced value on a conditional control flow path.
Leaks	InternalLeak<var_type>::<var_name>	A pointer is allocated locally but is never returned, assigned to an out-parameter, or routed to a sink.
	CondLeak<var_type>::<var_name>	A pointer is allocated locally but is never returned, assigned to an out-parameter, or routed to a sink on a conditional control flow path.
Safe	MemoryNeutral	The function completes its execution without altering the memory state of its caller.

Table 3-1 Tagging Reference

3.1.5 Applied Tagging Examples:

The following segments demonstrate how specific C functions map to their corresponding memory tags. For each example, the generic tag syntax (referenced from Table 3.1) is provided above the code snippet, while the exact tag generated by Leak Logic Mapper is displayed below it.

Sources:

AllocSource<var_type>::Ret

```
int* allocate_integer() {  
    // A new pointer is allocated  
    int* new_ptr = (int*)malloc(sizeof(int));  
  
    // The pointer is unconditionally returned  
    return new_ptr;  
}
```

AllocSource<int*>::Ret

CondAllocSource<var_type>::Ret

```
int* conditionally_allocate_integer(bool should_allocate) {  
    // A new pointer is conditionally allocated  
    if (should_allocate) {  
        int* new_ptr = (int*)malloc(sizeof(int));  
        // The allocated pointer is returned  
        return new_ptr;  
    }  
  
    // Control flow path where no allocation occurs  
    return NULL;  
}
```

CondAllocSource<int*>::Ret

AllocSource<var_type>::Arg{dstIdx}

```
void allocate_integer_out(int** out_ptr) {  
    // A new pointer is allocated  
    // It is unconditionally assigned to the out-parameter  
    *out_ptr = (int*)malloc(sizeof(int));  
}
```

AllocSource<int*>::Arg{0}

CondAllocSource<var_type>::Arg{dstIdx}

```
void conditionally_allocate_integer_out(int** out_ptr, bool should_allocate) {  
    // A new pointer is conditionally allocated  
    if (should_allocate) {  
        // The newly allocated pointer is assigned to the out-parameter  
        *out_ptr = (int*)malloc(sizeof(int));  
    }  
    // If should_allocate is false, the out-parameter is left unmodified  
}
```

CondAllocSource<int*>::Arg{0}

Figure 3-1: Source Tag Examples

Sinks:

FreeSink<var_type>::Arg{srcIdx}

```
void free_integer(int* ptr_to_free) {  
    // The pointer passed as argument index 0 is freed  
    free(ptr_to_free);  
}
```

FreeSink<int*>::Arg{0}

CondFreeSink<var_type>::Arg{srcIdx}

```
void conditionally_free_integer(int* ptr_to_free, bool should_free) {  
    // The pointer passed as argument index 0 is conditionally freed  
    if (should_free) {  
        free(ptr_to_free);  
    }  
}
```

CondFreeSink<int*>::Arg{0}

Figure 3-2: Sink Tag Examples

Passthroughs:

PassThrough<var_type>::Arg{idx}ToRet

```
int* process_and_return(int* input_ptr) {  
    // A pointer enters via argument index 0  
  
    // The pointer unconditionally exits via the return statement  
    return input_ptr;  
}
```

PassThrough<int*>::Arg{0}ToRet

CondPassThrough<type>::Arg{idx}ToRet

```
int* conditionally_return_pointer(int* input_ptr, bool should_return) {  
    // A pointer enters via argument index 0  
  
    // The pointer conditionally exits via the return statement  
    if (should_return) {  
        return input_ptr;  
    }  
  
    // Control flow path where the pointer is not returned  
    return NULL;  
}
```

CondPassThrough<int*>::Arg{0}ToRet

PassThrough<type>::Arg{srcIdx}ToArg{dstIdx}

```
void assign_pointer_to_out_param(int* input_ptr, int** out_dest) {  
    // A pointer enters via argument index 0  
  
    // It is unconditionally written into the memory address provided by  
    // argument index 1  
    *out_dest = input_ptr;  
}
```

PassThrough<int*>::Arg{0}ToArg{1}

CondPassThrough<type>::Arg{srcIdx}ToArg{dstIdx}

```

void conditionally_assign_pointer_to_out_param(int* input_ptr, int** out_dest,
bool should_assign) {
    // A pointer enters via argument index 0

    // It is conditionally written into the memory address provided by argument
index 1
    if (should_assign) {
        *out_dest = input_ptr;
    }

    // If should_assign is false, the out-parameter is left unmodified
}

```

CondPassThrough<int*>::Arg{0}ToArg{1}

Figure 3-3: PassThrough Tag Examples

Reference Losses:

ReferenceLoss<var_type>::Arg{srcIdx}

```

void wipe_reference(int **ptr) {
    // A double pointer enters via argument index 0

    // The pointer's dereferenced value is unconditionally overwritten
    *ptr = NULL;
}

```

ReferenceLoss<none>::Arg{0}

CondReferenceLoss<var_type>::Arg{srcIdx}

```

void conditionally_wipe_reference(int **ptr, bool should_wipe) {
    // A double pointer enters via argument index 0

    // The pointer's dereferenced value is conditionally overwritten
    if (should_wipe) {
        *ptr = NULL;
    }

    // Control flow path where the reference is left intact,
    // but the memory is never safely passed out or returned
}

```

```
CondReferenceLoss<none>::Arg{0}
```

Figure 3-4: Reference Loss Tag Examples

Leaks:

InternalLeak<type>::<var_name>

```
void process_and_leak() {  
    // A pointer is allocated locally  
    int* local_buffer = (int*)malloc(sizeof(int));  
  
    // The pointer is never returned, assigned to an out-parameter, or routed to  
    a sink before the function exits  
}
```

InternalLeak<int*>::<local_buffer>

CondInternalLeak<type>::<var_name>

```
void conditionally_leak_memory(bool error_condition) {  
    // A pointer is allocated locally  
    int* local_buffer = (int*)malloc(sizeof(int));  
  
    // Control flow path where the pointer misses a sink, return, or out-  
    parameter  
    if (error_condition) {  
        return;  
    }  
  
    // Normal execution path where the pointer is safely routed to a sink  
    free(local_buffer);  
}
```

CondInternalLeak<{int*}>::<local_buffer>

Figure 3-5: InternalLeak Tag Examples

Safe:

MemoryNeutral

```
void update_value(int* target_ptr, int new_value) {
```

```

// The pointer is dereferenced to modify the value it points to

// The function completes its execution without allocating new memory,
// freeing the pointer, or transferring its ownership elsewhere
if (target_ptr != NULL) {
    *target_ptr = new_value;
}
}

```

MemoryNeutral

Figure 3-6: MemoryNeutral Tag Example

Demonstrated is a program that utilises two deeply nested wrapper functions for allocation and deallocation. When memory operations cross a function boundary, the system prevents decay by propagating relevant tags upward through encapsulating functions. In this demonstration, the AllocSource and FreeSink tags are bubbled up. When evaluating the main function, the LLM identifies that its first operational call acts as an allocation source and its second acts as a deallocation sink. The LLM declares a pointer p is sourced and that same pointer p is freed. The orchestrator applies a cross referencing validation check and generates the MemoryNeutral tag. Ultimately, because no function in the call graph was assigned an InternalLeak tag, the system reports that the codebase is memory-safe.

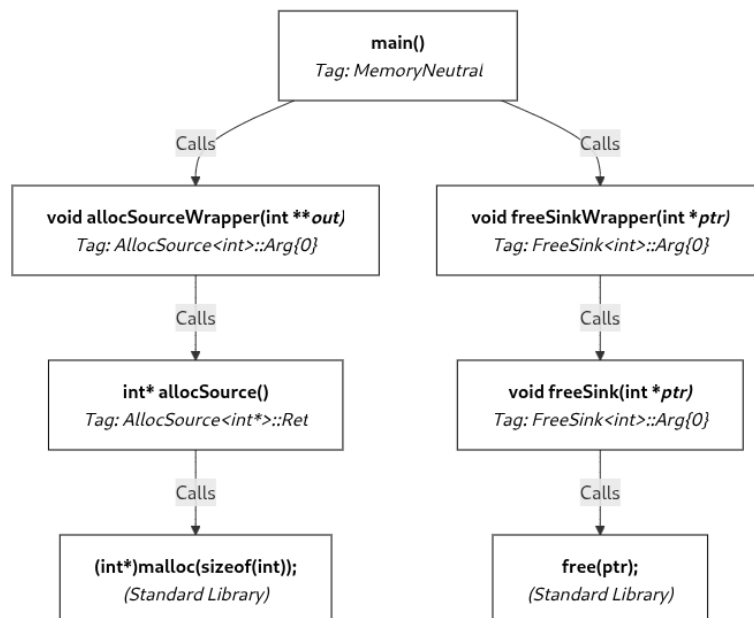


Figure 3-7: Nested sources and sinks (Flowchart)


```

#include <stdlib.h>

// Allocator Source
// Tagged as: AllocSource<int*>::Ret
int* allocSource() {
    return (int*)malloc(sizeof(int));
}

// Allocator Wrapper (using an out-parameter)
// Tagged as: AllocSource<int*>::Arg{0}
void allocSourceWrapper(int **out) {
    if (out != NULL) {
        *out = allocSource();
    }
}

// Free Sink
// Tagged as: FreeSink<int*>::Arg{0}
void freeSink(int *ptr) {
    if (ptr != NULL) {
        free(ptr);
    }
}

// Free Sink Wrapper
// Tagged as: FreeSink<int*>::Arg{0}
void freeSinkWrapper(int *ptr) {
    freeSink(ptr);
}

// Tagged as: MemoryNeutral
int main() {
    int *p = NULL;
    // Pass the address of p so the wrapper can modify it
    allocSourceWrapper(&p);
    // Pass the pointer to be freed
    freeSinkWrapper(p);
    return 0;
}

```

Figure 3-8: Nested sources and sinks (code)

While tags explicitly formalise memory contracts at a function's boundaries, they inherently abstract away intraprocedural implementation details. To preserve missing context, such as conditional triggers, the system relies on the aforementioned semantic summary. Together, they form a dual-layered architecture: tags define structural memory routing, while the

summary provides the analysis context necessary for the LLM to statically resolve complex internal control flow. This is particularly crucial for functions classified as MemoryNeutral. Although such functions lack boundary-crossing memory operations, they frequently evaluate conditions and return deterministic states, such as boolean flags, that govern wider interprocedural flow. By capturing these non-memory state transitions within the profile, the orchestrator ensures the LLM retains the exact context required to trace complex execution paths.

3.2 Prompting

To ensure reliable and reproducible analysis, Leak Logic Mapper employs a strictly architected system instruction. This prompt is divided into distinct operational sections, designed to enforce structural constraints and guide the LLM's semantic reasoning. The core components of this architecture are outlined as follows:

- **Persona and Objective Declaration:** This opening statement establishes the LLM as a deterministic static memory analyser; explicitly directing it to evaluate a provided target function and output a single, valid JSON object.
- **The Output Schema:** This serves as a comprehensive JSON template designed to standardise the LLM's output into strict data structures. The schema compels the LLM to categorise pointer states into the discrete domains: sources, sinks, passthroughs and reference losses. Upon detecting a heap operation, the LLM is instructed to generate a dedicated entry for the applicable domain. Ultimately, each entry encapsulates the complete lifecycle of a specific memory reference, documenting its point of origin, tracking its intraprocedural alias history, and, where applicable, defining the exact boundary (termed an exit portal within the system's architecture) through which it leaves the function's scope. Alongside these discrete domains, the schema mandates the generation of the semantic summary. This ensures that both structural memory routing and the semantic context are delivered simultaneously in a single, cohesive JSON object.
- **Definitions and Priority Rules:** This section focuses entirely on schema interpretation. It establishes strict heuristic guardrails to prevent semantic ambiguity. The schema

dictates the structure of the output, and priority rules govern how the model translates complex pointer interactions into the correct JSON keys.

- **Child Function Tag Rules:** To ensure deterministic inference, reference tags are injected alongside explicit semantic definitions. For example, the AllocSource and FreeSink tags are strictly mapped to dynamic heap allocations and deallocations, respectively.
- **Inter-procedural Control Flow Rules:** This section dictates that the LLM should evaluate conditional statements, identify dead code, and resolve deterministic control flow paths using the functional summaries of child memory profiles.
- **Semantic Extraction Rules:** This section is designed to prevent common LLM analysis pitfalls and hallucinations. It forces the strict separation of divergent control flow paths into distinct schema entries. It instructs the model to ignore standard NULL-guards for allocation statements and incoming pointers, and prohibits the LLM from making assumptions about the heap origin of incoming arguments, ensuring all pointer movements within the core function of the body are tracked as viable address references.
- **C-Language semantics:** To prevent the LLM from superimposing modern, memory-safe paradigms (such as Rust's move/ownership models (Rust Foundation, 2024)) onto the target code, this section explicitly grounds the model in standard C-language semantics. It enforces the rule that passing or assigning a pointer to another variable merely creates an alias of that memory address. It explicitly dictates that aliasing does not consume or invalidate the original pointer, thereby preserving its ability to be safely freed or utilised later in the execution path.
- **Execution Protocol:** The prompt concludes with an instruction to utilise an analysis scratchpad field before declaring schema entries. By forcing the model to articulate its intermediate reasoning steps prior to generating the final data structure, the system leverages Chain-of-Thought (CoT) prompting (Wei, J et al., 2022). This technique drastically reduces analytical hallucinations and ensures logical accuracy when tracing complex pointer states.

Introducing these constraints requires a careful balance to avoid "prompt bloat". An excessively long system prompt dilutes the model's attention mechanism, causing it to drop critical instructions as it attempts to satisfy numerous minor rules. Bloated prompts further incur heavy

token overhead, driving up inference latency and API costs. The prompt should not rely on exhaustive enumeration; it should be generalised as to avoid overfitting its instructions to known edge cases. Instead, edge cases are to be handled by the Python orchestrator, which can validate schema entries against known truths, and discard discrepant entries before formalising a final memory profile.

3.3 Deterministic Boundary Extraction (AST)

Before the system categorises any claims generated by the LLM, it establishes a deterministic ground truth for each function. Utilising the tree-sitter-c parser, the system traverses the AST to extract the absolute structural boundaries of a function's interface. This phase guarantees that the system operates within the strict realities of the source code.

Acting as a primary defense against model hallucination, the extracted AST data is collated as part of a validation context dictionary which is checked against the claims that the LLM makes before tags are generated. Two primary boundaries are extracted:

- **Parameters:** The parser traverses the function's signature to build a structured dictionary of all incoming parameters. For every parameter, the system extracts its integer position (index), its complete type signature, and its declared identifier.
- **Returns:** The parser traverses the function body to extract every explicit return statement. Because return statements frequently encapsulate complex expressions, type-casts, or pointer arithmetic, the AST must capture the exact literal string of the expression (e.g., `return (void*)p`).

This establishes the exact input and exit pathways available for a memory reference to enter or escape the function safely. If the LLM claims a locally allocated pointer was assigned to an out-parameter, a check is performed to ensure the claimed identifier represents a valid parameter within the function's signature.

3.4 Tag Generation

Prior to tag generation, the populated schema entries are filtered through a Routing Matrix. This architectural component dynamically assigns specific structural validation rules based on the heap operation's domain and its entry/exit boundaries.

When an object from the is extracted from the populated schema, the system first constructs a routing key for that claim. If the LLM claims that memory was allocated locally and escaped the function via a return statement, the system forms a key, representing these transactions, to query the routing Routing Matrix. This returns a TagRoute object containing the relevant tag template (`AllocSource<var_type>::Ret`) and a specific list of validator functions that must be satisfied for the claim to be accepted.

Before a formal tag is generated, the claim must pass every validator function assigned to its route. These validator functions cross-reference the LLM's claim against the validation context established in the AST extraction phase.

For example, the routing coordinate mentioned above triggers the following validators:

- **require_allocator:** Verifies that the exact line of C code claimed to allocate memory contains a known allocating function registered in the system's global registry. This may be a standard library call such as `malloc`, or a previously evaluated function which has been assigned the `AllocSource` tag.
- **exit_alis_in_returns:** Accesses the final alias of a pointer and checks if this identifier can be found in the exact line of C code for a return statement.

If a claim fails any assigned validator, it is acknowledged as a hallucination and immediately discarded.

If the object successfully passes all validators, a tag is formalised using the template provided by the TagRoute. During generation, the system dynamically injects relevant metadata into the template, this can include the variable type and parameter indexes. Finally, if the LLM's semantic analysis determined that the memory operation was only performed in a specific control flow path, the system prepends a `Cond` prefix to this tag.

3.5 Concurrent Execution

Due to the inherent latency when querying Large Language Models, the system is engineered to process the function call graph concurrently. Execution relies on Python's native `concurrent.futures` module and utilises the `ThreadPoolExecutor` class.

Execution begins by dispatching worker threads for all absolute leaf nodes. Each worker thread is responsible for submitting a query to the LLM, waiting for the response, filtering that response through the validation layer and returning a final memory profile. The Routing Matrix is defined as a global constant, and the worker threads execute pure functions that do not modify any shared global state. Because workers do not mutate the orchestrator's state, the architecture inherently prevents race conditions, entirely eliminating the need for complex mutex locks.

The main orchestrator thread utilises `concurrent.futures.wait` with the *FIRST_COMPLETED* parameter such that as soon as a worker thread returns its generated memory profile, that thread is released. The orchestrator takes exclusive responsibility for inserting the profile into a master dictionary mapping the entire codebase, and decrementing a dependency count for any parent functions waiting on that child.

The moment a parent function's dependency counter reaches zero, the system collates the validated memory profiles of all its children into a context payload and a new worker thread is queued for that parent. The degree of concurrency is scalable via a *MAX_WORKERS* environment variable, allowing the system to adapt according to the host machine's hardware capabilities.

3.6 Command-Line Interface and Diagnostic Reporting

The system is deployed as a Command-Line Interface (CLI) tool. During execution, the CLI actively logs the progression of the analysis pipeline. Upon the successful evaluation of all functions within the target codebase, the system outputs a comprehensive diagnostic report to the terminal. The report is structured into two distinct sections to provide both granular insights and high-level alerts:

- **Detailed Function Analysis:** The report first presents a breakdown of every analysed function, displayed sequentially in the order of their completed analysis. For each function, the system outputs its full memory profile generated by the orchestrator.
- **Vulnerability Summary:** To ensure critical security flaws are easily identifiable, the report concludes with a directive vulnerability summary. The system scans the generated tags across the entire call graph specifically for leaks. If any leaks are detected, this section explicitly highlights the compromised function, the data type and the specific pointer alias causing the leak. Otherwise, the system safely reports that no memory leaks were found across the call graph.

```
[*] Reading target file: test.c

=== STAGE 1: FUNCTION BOUNDARY ANALYSIS ===
[*] Extracting deterministic AST Interface...

=== STAGE 2: LLM SEMANTIC EXTRACTION ===
[*] Verifying Local Environment...
[*] Model selected: gpt-4o
[*] Initializing topological sort
[*] Thread started for: main...
[*] Pipeline initiated with 1 leaf functions...
[+] LLM response recieved for 'main' (Took 4.19s)
[+] Analysis complete: main

=== PIPELINE COMPLETE ===

[+] Topological Analysis Order (Bottom-Up):
main

=== DETAILED FUNCTION ANALYSIS ===

=====
FUNCTION: main
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The function 'main' allocates memory for an integer using 'malloc' and assigns it to the local variable 'ptr'. However, the allocated memory is not freed or passed out of the function, resulting in a memory leak. The function returns the integer value '1', which is unrelated to the allocated memory. There are no conditional blocks or child function calls in this code.",
  "tags": [
    "InternalLeak<none>::<ptr>"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'main'
    -> InternalLeak<none>::<ptr>

>> 1 leak(s) require attention.

=====
```

Figure 3-9: Exemplar terminal output for an isolated memory leak in a single function C program.

3.7 High Level Design Diagram

Figure 3.5 illustrates the high-level architecture and inter-procedural execution flow of the framework. The diagram specifically highlights the system's dynamic feedback loop, demonstrating how the orchestrator resolves child dependencies, constructs contextual prompts, and validates LLM outputs.

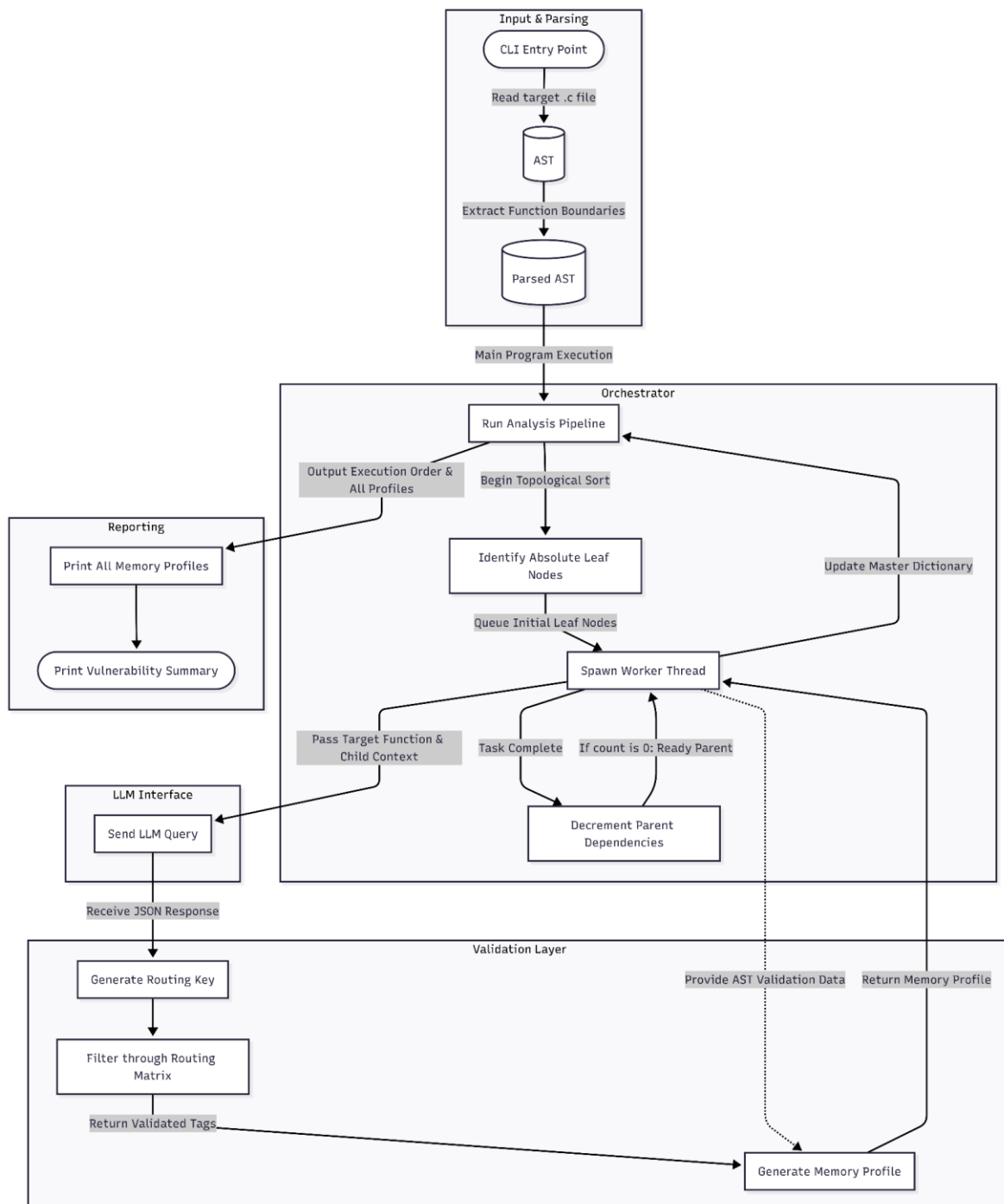


Figure 3-10: High Level Design Diagram

3.8 Architectural Modularity: Cloud API and Local Backends

To ensure the system remains scalable, future-proof, and adaptable to different security requirements, the core orchestration pipeline is deliberately decoupled from the underlying LLM provider. While the development and testing phases of this system specifically utilised GPT-4o, deployed via Microsoft Azure (Microsoft, 2026), the architecture avoids rigid dependencies by implementing the widely adopted OpenAI API standard (OpenAI, 2026) such that the system can easily pivot to alternative models or cloud providers. In addition, Ollama (Ollama, 2026) support has been integrated to address environments with strict data privacy requirements or restricted network access. The active backend is determined dynamically at runtime via user configured environment variables. As open-source models improve or new enterprise APIs become available, the system can pivot to a new provider without requiring any refactoring of the core pipeline.

3.9 System Scope (Minimum Viable Product Constraints)

Leak Logic Mapper has been designed as a proof-of-concept system to evaluate the efficacy of LLMs in tracking complex memory states. To adhere to a Minimum Viable Product (MVP), the system deliberately constrains its scope and employs minimal AST parsing. The following C programming features fall outside the development scope of Version 1, however they represent intentional architectural boundaries rather than inherent failures of the LLM-driven approach.

- **Single File Constraint:** Inter-procedural analysis is restricted to a single .c file. Resolving dependencies across externally linked libraries is deferred as a preceding linker phase is not implemented.
- **Macros, typedefs and Conditional Directives:** The system parses raw source code without executing a preprocessor pass, meaning macros, typedefs and directives (e.g., `#ifdef`) are unresolved. However, a macro with an intuitive name, such as `ALLOC` (see section 4.1.6), may afford sufficient context for an LLM to infer its underlying behaviour. Because the system's minimum viable scope delegates the interpretation of these opaque elements entirely to the LLM's probabilistic reasoning, rather than programmatic resolution, their exact evaluation cannot be assured. Therefore,

unexpanded macros and typedefs remain a source of inherent unpredictability within the framework.

- **Field-Sensitive Analysis:** Escaping or direct allocations to struct members are not currently accommodated. The system does not parse struct definitions from the AST; consequently these are excluded from the context injection. The *var_type* field in the tagging templates has not been accommodated to format the pointer member access operator.
- **Offset Tracking and Pointer Arithmetic:** Mathematically modified offset pointers are not mapped. As with struct members, the *var_type* field has not been formatted to accommodate for pointer offsets.
- **Nested Heap Structures:** The tagging system does not currently accommodate tracking multidimensional arrays or arrays of pointers across inter-procedural boundaries. The JSON schema has not been tailored to allow for tracking nested memory states, meaning that if an array of pointers escapes the local scope, the system maps the transfer of the base container but loses tracking fidelity for its internal elements.
- **Top-Down Data Flow:** To maintain concurrent processing, the orchestrator utilises a strict bottom-up evaluation. This makes the system context-insensitive; target functions are evaluated without parent context and are inherently unaware of the specific arguments passed by their callers. Without parsing and injecting parent context, dynamic dispatch utilising function pointers is unresolved.
- **Recursion:** The bottom-up topological sort requires a strictly acyclic call graph. Advanced static analyzers resolve cyclic memory states via fixed-point iteration, repeatedly evaluating the cycle until the memory footprint stabilises (Calcagno et al., 2011). While theoretically viable for LLMs, this requires establishing strict heuristics to bound the required prompt iterations and prevent runaway API overhead. Therefore, the MVP actively detects recursive chains and raises a `CycleDependencyError` to safely halt evaluation and prevent false tagging.
- **Global and Static Variables:** The system evaluates memory contracts by strictly monitoring the explicit input and output boundaries of a function's signature. Because global and static variables bypass these boundaries validating them requires whole-program state monitoring. To maintain the system's lightweight, function-isolated parsing, this out-of-band state resolution is designated as out of scope.

- **Strict Spatial Targeting:** Leak Logic Mapper strictly evaluates spatial memory leaks (missing sinks). Temporal errors, such as Use-After-Free, Double Free, and Dangling Pointers, represent a fundamentally different class of vulnerability and fall outside the system's development scope.

See Future Works (6.3) for more information.

4. Implementation

This chapter details the development and implementation process for Leak Logic Mapper, detailing the translation from a conceptual architecture into a fully operational Python application.

4.1 Project Setup

4.1.1 System Setup and Core Technologies

Leak Logic Mapper requires a focussed set of third-party dependencies to facilitate its analysis pipeline. The core libraries include: tree-sitter-c, openai and python-dotenv (Kumar, 2026).

To ensure the system remains modular and adaptable to different security or hardware environments, all system setup is configured by the user via a .env file. A foundational .env.example file (shown in Figure 4.1) is distributed with the software alongside comprehensive setup instructions included in a README Markdown file.

```
# Choose your backend: (api | local)
LLM_BACKEND=api

# API BACKEND SETTINGS
# Required for all API backends
OPENAI_API_KEY=your_api_key_here
OPENAI_API_MODEL=your_model_here

# For Standard API Providers
# Uncomment the line below and add the provider's URL
# OPENAI_BASE_URL=your_url_here

# For Azure OpenAI
# Uncomment and fill these ONLY if you are using an Azure account
# OPENAI_ENDPOINT=your_endpoint_here
# OPENAI_API_VERSION=2024-02-01

# LOCAL BACKEND SETTINGS
LOCAL_MODEL=your_model_here
LOCAL_PORT=11434

# ADVANCED SYSTEM SETTINGS
```

```

# Number of threads for parallel analysis
MAX_WORKERS=4
# Max time to wait for API response (in milliseconds)
HTTP_TIMEOUT_MS=60000

# Backoff and Retry
MAX_RETRIES=3
BASE_DELAY=3

```

Figure 4-1: .env example file.

File Structure: Leak Logic Mapper’s implementation is segmented into distinct scripts.

Figure 4.2 illustrates the directory structure, providing context for the modules discussed throughout this chapter.

```

Leak_Logic_Mapper/
├── .env.example           # Template for user-defined configuration and API keys
├── main.py               # Application entry point and CLI argument parser
├── README.md             # Comprehensive setup and execution instructions
├── requirements.txt       # Explicit list of third-party Python dependencies
├── src/                  # Core application logic
│   ├── api_interface.py  # OpenAI/Azure cloud LLM integration
│   ├── ast_parser.py     # Tree-sitter deterministic boundary extraction
│   ├── constants.py      # Shared programmatic constants
│   ├── exceptions.py     # Custom error handling
│   ├── local_interface.py # Offline Ollama LLM integration
│   ├── orchestrator.py   # Call graph generation and concurrent thread management
│   ├── profiler.py       # Evaluates functional boundaries and applies tag routes
│   ├── reporter.py       # Prints diagnostic output and vulnerability summaries
│   ├── system_instruction.md # The prompt injected into the LLM
│   ├── utils.py          # Data normalisation and logging utilities
│   └── validators.py     # The Routing Matrix and algorithmic validation functions
└── tests/
    └── sample_test_cases/ # A suite of C files used for testing and validation

```

Figure 4-2: Directory structure

4.1.2 Deterministic Parsing with Tree-Sitter

Tree-sitter Queries: Parsing code with the tree-sitter-c library is performed by matching specific queries against nodes in the AST. A node is a fundamental building block of an AST that represents a single construct from the source code, such as a statement, expression or function definition. Queries are constructed using S-expressions: a notation used to describe tree patterns. By applying S-expressions, Leak Logic Mapper isolates function names, bodies, signatures, child calls, parameters, and return statements before any LLM inference occurs. The exact S-expression queries implemented in the `ast_parser.py` module are detailed below:

```
FUNC_QUERY = """
(function_definition # Look for the root node of a complete C function
definition
  type: ( ) @return_type # Store the return type
  declarator: (function_declarator # Look for the function declarator node
    declarator: (identifier) @function_name # Store the function name
  ) @function_signature # Store the function signature (used later to
access parameters)
) @function_body # Store the full function body

# Define a new query for functions that return pointers
(function_definition
  type: ( ) @return_type
  declarator: (pointer_declarator # Look for a pointer modifier
    declarator: (function_declarator
      declarator: (identifier) @function_name
    )
  ) @function_signature
) @function_body
) """

PARAMS_QUERY = "(parameter_declaration) @parameter"
CHILD_QUERY = "(call_expression function: (identifier) @child_name)"
RET_QUERY = "(return_statement ( ) @ret_statement)"
```

Figure 4-3: tree-sitter-c queries

The system is required to deliberately split function extraction into two distinct patterns. When a standard C function is parsed, the AST places a *function_declarator* node within the definition. If a function returns a pointer, the AST instead places a *pointer_declarator* node. A single expression to create a unified query was attempted but frequently resulted in “binding failures”, where capture tags, denoted by the @ symbol and used to access matches, failed to store the correct node. Once functions have been isolated, the system executes subsequent

queries to generate lists of parameters, function calls, and return queries against the captured function signature and function body, respectively. The *CHILD_QUERY* will at first match and store every function call made within the function body, including library functions. Before a final object storing all parsed functions is returned, any function call made within the body of a function that is not a user-defined function is removed.

Before the Python orchestrator can validate any deterministic boundaries generated by the AST against the probabilistic data extracted by the LLM, it must establish a unified data format. The raw AST data represents the literal syntax of the source code, and frequently contains syntactic artifacts such as redundant parentheses or inline type-casts. However, when the LLM populates the JSON schema, it is likely to declare a pointer by its raw identifier. Standard string-matching algorithms would inherently break under these conditions.

The base variable is the fundamental, unmodified alphanumeric identifier of the pointer. The base variable must adhere strictly to C-standard identifier rules: it must contain only alphanumeric characters and underscores, contain no whitespace, and be entirely stripped of all prefix and postfix operators. (ISO/IEC, 2011) The system performs lightweight normalisation steps to extract these base variables utilising two dedicated utility functions within *utils.py*.

The *normalise_parameters()* function takes a raw parameter string (e.g., *void **ptr*) and safely separates the base variable from its type declaration. This is performed by counting the pointer depth natively and splitting the string. This ensures that parameter boundaries can be verified against variable names, regardless of how the spacing or pointer asterisks were originally formatted.

Return statements are inherently more complex as they often encapsulate expressions. The *normalise_return_statement()* function systematically strips away redundant outer parenthesis, and uses Regular Expressions to safely remove the *return* keyword and basic inline type-casts. Finally it removes trailing whitespace to allow for pattern matching downstream.

Figure 4.4 displays a debug output demonstrating AST data extraction in comparison with LLM extraction. Notice the in-line type-cast and wrapping parentheses in the AST *rets* object, contrasted with the direct *exit_alias* and *exit_portal* in the LLM extraction.

```
>> Code:
char * allocator() {

    void * p = malloc(sizeof(char));

    void * alias_p = p;

    return (char*) (alias_p);
}

>> AST INTERFACE (Deterministic Boundaries):
{
  "return_type": "char*",
  "params": [],
  "children": [],
  "rets": [
    "(char*) (alias_p)"
  ]
}

>> LLM SEMANTIC PAYLOAD:
Sources: [
  {
    "allocation_statement": "void * p = malloc(sizeof(char));",
    "immediate_assignment": "local_variable",
    "tracked_pointer": "p",
    "variable_type": "void *",
    "alias_trace": [
      "p",
      "alias_p"
    ],
    "is_reference_lost": false,
    "exit_alias": "alias_p",
    "exit_portal": "return_statement",
    "is_conditional": false
  }
]
```

Figure 4-4: Return extraction from the AST and the LLM.

4.1.3 LLM Integration and Output Structure

The system employs dedicated interface modules `api_interface.py` and `local_interface.py` to manage backend connections. The integration is highly configurable, allowing a target model to be specified for a backend dynamically via the `.env` file. These modules are responsible for instantiating an OpenAI client object to initialise essential connection parameters such as base URLs, custom endpoints, timeout constraints and API keys where applicable.

To construct the inference query, the interface modules read the system instruction from a dedicated Markdown file: `system_instruction.md`. This instruction is combined with the target function and any dependent child function memory profiles to form a comprehensive prompt. Inference is then triggered via the `chat.completions.create` method.

During this request, the system enforces a JSON response format and the model's temperature parameter is set to 0.0. Setting the temperature to zero eliminates creative variance by forcing greedy decoding, wherein the model consistently selects the most probable next token. In the context of static analysis, this deterministic behaviour is critical; it mitigates the risk of hallucination and ensures that identical code inputs yield reproducible semantic extractions (OpenAI, 2026).

To ensure the pipeline against network instability and provider constraints, the API interface implements an exponential backoff and retry mechanism. This is triggered upon specific HTTP status codes indicative of server errors (Nottingham and Shapira, 2012) (Fielding et al, 2022):

- 429 - Rate limit exceeded
- 502 - Bad Gateway
- 503 - Servers overloading
- 504 - Gateway Timeout

If inference fails to recover after a user-defined maximum number of retry attempts, set with *MAX_RETRIES* in .env, the orchestrator halts the pipeline. The system captures the resulting error and rescues the partial analysis state, reporting the memory profiles of all functions that were successfully evaluated prior to failure.

To bridge the gap between LLM semantic extraction and programmatic evaluation, the implemented schema enforces rigid data types and specific string literals. This rigidity is critical; it guarantees that the LLM's response can be safely deserialised by the orchestrator using standard JSON parsing libraries. A representative fragment of the schema is demonstrated below:

```
"sources": [  
  {  
    "allocation_statement": "string (The exact line of C code for  
allocation)",  
    "immediate_assignment": "string (The left-hand side of the  
allocation_statement. Strictly: \"local_variable\", \"return_statement\",  
\"out_parameter\", or \"none\")",  
    "tracked_pointer": "string (The initial variable name holding the  
memory)",  
    "variable_type": "string (The C data type)",  
    "alias_trace": ["string (Chronological chain of aliases tracking
```

```

this specific address)"]],
    "is_reference_lost": "boolean (True ONLY if the address becomes
unreachable without being freed or passed out)",
    "exit_alias": "string (The final variable name holding the memory,
or \"none\" if the reference was lost)",
    "exit_portal": "string (Strictly: \"return_statement\", the literal
name of the destination argument for out_parameters, or \"none\")",
    "is_conditional": "boolean (True if this allocation/exit path is
skip-able)"
  }
],

```

Figure 4-5: JSON Schema Sources Object

Each key within a discrete domain serves a programmatic function. Keys may overlap between domains, but each domain has a unique implementation. The structural implementation for core keys is as follows:

- **Allocation Statement and Immediate Assignment:** Implemented for the sources domain, these keys ground the origin and immediate use of a pointer coming into the target function for an Allocating Source.
- **Tracked Pointer:** Identifies the initial name of the pointer for which the schema object has been generated.
- **Variable Type:** Specifies the C data type of the tracked pointer.
- **Alias Trace:** Records a chronological list of all local variable aliases that hold the original memory reference over the scope of the target function.
- **Exit Portal:** Implemented for the sources and passthroughs domains; it defines the specific boundary through which a pointer escapes the target function's scope.
- **Free Statement and Overwrite Statement:** Implemented for the sinks and reference losses domains respectively; these fields identify where the memory reference for the tracked pointer is safely terminated or destructively lost.
- **Is Conditional:** A boolean value that evaluates to true if the tracked pointer only satisfies the qualities of its respective domain on a specific control flow path.

All remaining keys act as validation values to catch edge cases, or to supplement CoT reasoning.

JSON keys are established in a centralised schema class within the python constants module. This module defines a series of constants which map directly to the discrete domains and specific pointer states that are outlined within the schema. This avoids the use of magic strings within the source code of the Python orchestrator.

```
class Schema:
    """
    Direct mapping to the JSON schema.
    """
    SOURCE = "sources"
    SINK = "sinks"
    PASSTHROUGH = "passthroughs"
    REFERENCELOSS = "reference_losses"
    LOCAL = "local_variable"
    RETURN = "return_statement"
    OUT_PARAM = "out_parameter"
    NONE = "none"
```

Figure 4-6: Schema Class

Supporting rules for the JSON schema provide guidance for extraction and population. Figure 4.7 dictates how the model should interpret tags. The complete system instruction definition is available for review in Appendix A.

```
CHILD FUNCTION TAG RULES:
1. "AllocSource": Treat as a new allocation (Source).
2. "FreeSink": Treat as a deallocation (Sink).
3. "PassThrough": Ownership is preserved and transferred. Do not log a
Reference Loss for pointers passed into a child with this tag.
4. "ReferenceLoss": Identity is destroyed. If a pointer enters a child
with this tag and no PassThrough tag is present for that same pointer,
set "is_reference_lost": true.
```

Figure 4-7: System Instruction - Child Function Tag Rules

4.1.4 Pipeline Execution:

Upon initiation of the primary execution pipeline, the orchestrator traverses the extracted AST function data to construct the directional links of the call graph. To facilitate this, two primary data structures are generated: a dependency counter, *pending_dependencies*, and an inverse call map, *parent_map*. For every child function invoked within a parent's function body, the parent's dependency counter is incremented. Simultaneously, the parent is appended to the *parent_map* using the child as the primary key.

Execution follows a concurrent adaptation of Kahn's algorithm (Kahn, 1962) to perform a bottom-up topological sort of all functions. When a newly evaluated memory profile is returned by a worker thread, it is securely stored within the shared *memory_profiles* dictionary using the function name as the primary key. The system then queries the *parent_map* and decrements the dependency counter by 1 for all parents awaiting on that specific child. For those parents, a check is made such that if the parent's pending dependency counter reaches zero, a new worker thread is dispatched to analyse the now-unblocked parent function.

Concurrent execution continues until all unblocked functions have been successfully processed. Because Kahn's algorithm requires a DAG to resolve completely, the dependency counter of any function with a cyclic dependency will never reach zero. Consequently, these functions are permanently blocked and never queued for analysis. This limitation inherently applies to self-recursive functions, which perpetually act as their own unresolved dependency.

The orchestrator performs a final reconciliation check once the pipeline halts. If the total number of populated memory profiles in the final dictionary is less than the total number of functions originally identified during the AST parsing phase, it indicates the presence of an unresolvable cycle. In this event, the system raises a *CycleDependencyError* to alert the user of the unsupported call graph.

4.1.5 Hallucination and Edge Cases:

A vulnerability of this architecture is that a heavily architected system prompt acts as a single point of failure. During development, adjustments risked degrading previously stable functionality, necessitating continuous regression testing. LLM inference is analogous to that of a grey box. The semantic summary and analysis scratchpad may show some visibility into the model's reasoning, and the JSON schema does confirm the exact payload fed back into the

system. However, it is fundamentally impossible to map a specific prompt alteration to its exact effect on the model's final output. To mitigate fragility, ad-hoc prompt modifications were restricted in favor of batching major updates in discrete development cycles.

LLM inference presented two major hurdles during development, hallucination and overfitting to edge cases. Frequent hallucinations ranged from redundant source objects, erroneous passthroughs and reference losses, to mispopulated objects. Edge cases frequently emerged in scenarios where non-standard pointer logic disrupted the model's ability to adhere to the JSON schema. For example, LLMs showed difficulty in populating schemas for pointer swap functions, frequently omitting passthrough objects needed to generate passthrough tags. Here the semantic summary effectively mitigated this failure: if no tags were able to be generated, but the description included the key details necessary for inter-procedural analysis, evaluation of the parent function tended to hold firm.

The primary challenge involved balancing the necessity for a prompt explicit enough to populate the correct schema object accurately, while keeping the directions generic enough such that they were not overfit to a specific test case. The schema had to act as a guide; it could not be directly patched for a hallucination found in a specific test case. There was a risk that introducing specific examples to patch a test case would inadvertently over-condition the LLM, causing it to apply rules only to exact matches and failing to understand the transferable semantics behind that operation. As such, explicit examples in the schema were minimised. Broader guiding principles were defined in the *SEMANTIC EXTRACTION RULES* section of the schema and served to resolve conditions leading to hallucination and edge cases. As an example, the *NEVER EXTRACT CHILD OPERATIONS* rule states that when building sources, sinks, and reference loss objects, only statements within the target function's code block were to be extracted into the object. This prevented cases where the LLM had observed a malloc or free call within the semantic summary of the child function, and had mispopulated the object from the parent function with the exact call detailed in the summary.

4.1.6 Validation and Tag Routing

Despite the efficacy of generalised semantic extraction rules, the inherent opacity of LLM inference dictated that hallucinatory side effects could not be entirely eradicated. Coupled with a development mandate restricting ad-hoc prompt modifications to preserve system stability, prompt engineering alone was insufficient for system accuracy. The dual constraint necessitated post-inference error handling.

Tag routes, found in `utils.py`, are dataclasses mapping a tag formatting template and a list of validating functions to an object in the schema. The route key is generated by invoking a call to a lambda function. For every domain in the schema, the lambda function returns a tuple containing at minimum an identifier for the relevant domain. For domains tracking a pointer that may exit a local scope, additional contextual attributes are required to formulate a precise routing coordinate: `AllocSource` keys include both the *immediate_assignment* type and the *exit_portal*, `PassThrough` keys include the *exit_portal*.

Tag routes are designed to enforce type-safety regarding the *immediate_assignment* and *exit_portal* keys. These keys contain strict value assignment instructions, incorporated into the schema itself. Under this, these fields in the schema should always be populated with an expected value type, even in the event that that value is incorrect. *Immediate_assignment* is defined as: “*The left-hand side of the allocation_statement. Strictly: local_variable, return_statement, out_parameter, or none*”. *Exit_portal* is defined as: “*Strictly: return_statement, the literal name of the destination argument for out_parameters, or none*”.

As an *exit_portal* is allowed to be populated as a literal identifier, a preprocessing step is performed within the `_evaluate_boundaries()` function in `profiler.py` to translate this into the value `OUT_PARAM`. A check is made such that if the identifier cannot be matched in the parameter list obtained during AST extraction, this will be treated as a hallucination and translated into the value `NONE`. Because resolving the *exit_portal* inherently requires AST cross-referencing, this function opportunistically validates related boundary conditions. It extracts parameter indexes (`src_idx`, `dst_idx`) and verifies return statements. Performing this comprehensive check upfront prevents the downstream validator functions from having to repeat these validations.

A tag route was written for all combinations of *immediate_assignment* type and *exit_portal* observed during development; a tag route allows resolution of both correct information and

hallucinations to validator functions, in the event that one of these fields is illogically mispopulated an illogical tag route can still execute the correct control flow (see SOURCE, OUT_PARAM, NONE in figure 4.8).

Tag routes were engineered with a focus on modularity. As further development cycles ensued and a tag route needed amending, or a new tag route was devised, it could be simply added as a new entry in the routing matrix. Crucially, validating functions are defined outside of the route matrix, such that the tag route need only contain a list of Callable functions as opposed to actual implementations. All validating functions return to a boolean value, taking the JSON object returned from the LLM and the deterministic boundary data parsed from the AST to perform a validation check. This allows for reuse of a validator function across multiple tag routes.

Sources:

The source object is used to generate both AllocSource and InternalLeak tags. A source object generates a route key with the identifier *Schema.SOURCE*, the *immediate_assignment* value from the schema, and the *exit_portal* value from the schema respectively.

Route Key (SOURCE, <i>immediate_assignment</i> , <i>exit_portal</i>)	Use Case	Tag Template	Validators
SOURCE, LOCAL, RETURN	A source is assigned to a pointer and exits via a return statement.	{prefix}AllocSource <{var_type}>::Ret	require_allocator, exit_alias_in_rets
SOURCE, LOCAL, OUT_PARAM	A source is assigned to a pointer and exits via an out parameter.	{prefix}AllocSource <{var_type}>::Arg{ {{dstIdx}}}	require_allocator
SOURCE, RETURN, RETURN	A source exits immediately as the allocating call is in line with a return statement.	{prefix}AllocSource <{var_type}>::Ret	require_allocator, is_immediate_return
SOURCE, OUT_PARAM, OUT_PARAM	A source is immediately assigned to an out	{prefix}AllocSource <{var_type}>::Arg{	require_allocator

	parameter.	{{dstIdx}}	
SOURCE, LOCAL, NONE	A locally allocated pointer never exits the local scope.	{prefix}InternalLeak<{var_type}>::<{var_name}>	require_allocator, is_not_freed
SOURCE, OUT_PARAM, NONE	Error Catch: in the case that the LLM hallucinates that a locally allocated pointer was immediately assigned to an out_param, but in reality never exits the local scope.	{prefix}InternalLeak<{var_type}>::<{var_name}>	require_allocator, is_not_freed
SOURCE, NONE, NONE	An allocating call is never assigned to a pointer.	{prefix}InternalLeak<{var_type}>::OrphanedReturn	require_allocator, is_source_allocation_statement_in_code

Table 4-1 Sources Tag Routes.

In the following output, the *allocation_statement* contains the `ALLOC` macro, a standard construct utilised within the Juliet Test Suite to map to the stack-allocator `alloca`. Because this macro provisions memory on the stack rather than the heap, classifying it as a dynamic allocation source represents an LLM hallucination. Tracing this, as the *immediate_assignment* is a *local_variable* and the *exit_portal* is none, the system would initially generate the key `SOURCE, LOCAL, NONE`. This key maps to the tag template: `{prefix}InternalLeak<{var_type}>::<{var_name}>`. This routing key has two validators: *require_allocator* and *is_not_freed*. *require_allocator* checks the registry of known allocators, which is predefined with: `malloc`, `calloc`, `realloc`, `wcsdup`, and `strdup`, and dynamically appends the name of any child function with an `AllocSource` tag. As `ALLOC` is not in this registry, the check evaluates to false. *is_not_freed* checks if there is an associated sinks object tracking the pointer in the source object; no such object exists and this check also evaluates to false. For the tag to be generated, all validating checks must evaluate to true; therefore, the tag is correctly rejected.


```

>> LLM SEMANTIC PAYLOAD:
Sources: [
  {
    "allocation_statement": "data = (char *)ALLOCA(100*sizeof(char));",
    "immediate_assignment": "local_variable",
    "tracked_pointer": "data",
    "variable_type": "char **",
    "alias_trace": [
      "data"
    ],
    "is_reference_lost": false,
    "exit_alias": "data",
    "exit_portal": "none",
    "is_conditional": false
  }
]
Sinks: []
PassThroughs: []
ReferenceLosses: []

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates memory on the stack using `ALLOCA` and initializes it with a string. The memory is used within the function and automatically deallocated when the function exits. The first `if` block is dead code due to the constant `staticFalse`, and the second `if` block executes unconditionally due to the constant `staticTrue`. No explicit deallocation is performed, but the stack allocation ensures proper cleanup.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Figure 4-8: ALLOCA Validation Check

Sinks:

Sink objects are constrained to a single tag, utilising *Schema.SINK* as the unique route key. A validation check verifies that the tracked pointer exists within the parameter list extracted during AST analysis.

Route Key (SINK)	Use Case	Tag Template	Validators
SINK	A pointer parameter is freed	{prefix}FreeSink<{var_type}>::Arg{{{srcIdx}}}	Require_deallocator, is_mutating_argument

Table 4-2: Sinks Tag Routes.

PassThroughs:

There are two possible tags for PassThrough objects, representing exits via a return statement or an out parameter. A route key is generated with the identifier *Schema.PASSTHROUGH* and the *exit_portal*.

Route Key (PASSTHROUGH, exit_portal)	Use Case	Tag Template	Validators
PASSTHROUGH, RETURN	A pointer argument exits via a return statement.	{prefix}PassThrough <{var_type}>::Arg{ {{srcIdx}}}ToRet	entry_arg_is_not_eq ual_to_exit_portal
PASSTHROUGH, OUT_PARAM	A pointer argument exits via an out parameter.	{prefix}PassThrough <{var_type}>::Arg{ {{srcIdx}}}ToArg{{ {dstIdx}}}	entry_arg_is_not_eq ual_to_exit_portal

Table 4-3: PassThrough Tag Routes

ReferenceLosses:

There is only one tag that can be generated from a ReferenceLoss object, a route key is generated with the identifier *Schema.REFERENCELOSS*. The same validator function to check applied to sinks to check if the tracked pointer exists in the parameter list is made.

Route Key (REFERENCELOSS)	Use Case	Tag Template	Validators
REFERENCELOSS	A pointer parameter is destructively overwritten.	{prefix}ReferenceLo ss<{var_type}>::Arg {{{srcIdx}}}	is_mutating_argume nt

Table 4-4: ReferenceLoss Tag Routes

Full implementations for all Validator functions can be found in Appendix B.

On Reference Losses:

Reference Losses were implemented during late development, as a way to acknowledge destructive memory overwrites of a pointer before the safe handling of said memory reference. Testing revealed that the LLM frequently populates this object with locally-scoped reference losses, such as memory leaked via early return statements. While semantically correct, local reference losses were not required for inter-procedural tagging goals, and were not applied to internal leak tagging routes as this logic was already handled reliably via comparison checks between source and sink objects.

```
>> Code:
void target_function(int error_flag) {
    int* buffer = malloc(1024 * sizeof(int));
    if (buffer == NULL) return;

    printf("[DEBUG] Allocated buffer at %p\n", (void*)buffer);

    if (error_flag == 1) {
        return;
    }

    free(buffer);
}
```

```
ReferenceLosses: [
  {
    "tracked_pointer": "buffer",
    "variable_type": "int*",
    "alias_trace": [
      "buffer"
    ],
    "overwrite_statement": "return;",
    "overwritten_with": "none",
    "is_conditional": true
  }
]
```

Figure 4-9: Local Scope Reference Loss

An analysis of this over-population provides a practical demonstration of the 'grey box' analogy discussed earlier. During development, an ad-hoc prompt modification was tested, explicitly instructing the model to log reference losses solely for incoming pointer parameters. However, this quick patch proved ineffective; the LLM continued to accurately, yet unhelpfully, to flag local memory leaks as lost references. This demonstrates a criticality regarding the importance of naming for semantic extraction; the term 'ReferenceLoss' carries strong associations with general memory leaks such that the object's literal name is conflicted with the targeted prompt instructions. The validation framework served as a structural safeguard here; noise in the JSON schema is filtered out by tag routes, ensuring there was no regression when generating tags

from the previously established domains and preventing false tags being generated from data noise.

4.2 Testing

Leak Logic Mapper has been tested iteratively during development.

The following section evaluates Leak Logic Mapper's performance across a selection of custom test case scenarios. To confirm the validity of these test cases, they are cross-referenced against established dynamic analysis tools: AddressSanitizer (ASan) (Serebryany *et al.*, 2012) and Valgrind Memcheck (Nethercote and Seward, 2007). All test cases were compiled using the GNU Compiler Collection (GCC) (Free Software Foundation, 2026). The `-g` (`--debug`) flag was used universally to include debug symbols, ensuring that both dynamic tools report memory errors with human-readable file names and line numbers. Because these tools intercept memory allocations using fundamentally different mechanisms, they cannot analyse the same executable; each test case was compiled into two distinct binaries: one compiled strictly with the `-g` flag for Valgrind analysis, and a second compiled with both `-g` and `-fsanitize=address` to instrument the binary specifically for AddressSanitizer.

To maintain the integrity of the evaluation, test case functions were stripped of comments to prevent providing the LLM with unintended contextual cues. While inline documentation is a primary asset for semantic extraction in production environments, its exclusion here ensures the model is assessed solely on its ability to parse raw code logic. Comments outside of a function's local scope are not parsed by the AST and as such not fed to the LLM. There is no traditional way to verify that comments outside of a function's scope actually represent crucial information pertaining to that function. It may be possible to have performed this check with an LLM, though performing this extraneous preprocessing step would incur prohibitive computational overhead.

Due to space constraints, the full suite of test cases and their raw outputs are cataloged in Appendix C. A representative subset highlighting baseline functionality, iterative refinements, and observed edge-case behaviors is analysed below.

These test cases were inspired by the Juliet Test Suite and are distributed with Leak Logic Mapper.

Test ID:	Test Scenario	Found In	Status
01	Basic Allocation	C.01	Baseline Pass
02	Missing Free	4.2.1	Baseline Pass
03	Pointer Swap	C.03	Minor Variance
04	PassThrough Return	C.04	Minor Variance
05	PassThrough Out Parameter	C.05	Minor Variance
06	Orphaned Return	C.06	Baseline Pass
07	Overwritten Return	C.07	Baseline Pass
08	Conditional Free	4.2.2	Refined Pass
09	Early Return	C.09	Baseline Pass
10	Alloca	4.2.3	Future Work Required
11	Deterministic Control Flow	C.11	Baseline Pass
12	Conditional Leak	4.2.4	Baseline Pass
13	Reference Overwrite	C.13	Baseline Pass
14	Deep Nesting	C.14	Baseline Pass

Table 4-5: Test Case Overview

Baseline Pass - Leak Logic Mapper correctly identified all tags and logic on the first attempt.

Refined Pass - Leak Logic Mapper correctly identified all tags and logic following iterative bug fixing.

Minor Variance - Leak Logic Mapper correctly identified core logic but exhibited minor deviations in tag accuracy.

Future Work Required - Identified limitations beyond current scope.

4.2.1 - Missing Free

Code

```
/*
 * TEST CASE: This test demonstrates how the system handles a pointer that is not
 * freed.
 *
 * EXPECTED TAGS: [create_session()] AllocSource<void*>::Ret
 *                [end_session()] FreeSink<void*>::Arg{0}
 *                [target_function()] InternalLeak<void*>::<current_session>
 *
 * VULNERABILITY: The programmer forgets to call end_session() before the program
 * terminates.
 */

// DEPENDENCY FUNCTIONS
// Safe AllocSource
void* create_session(size_t size) {
    printf("[DEBUG] Allocating %zu bytes...\n", size);
    return malloc(size);
}

// FreeSink
// Safe FreeSink that is never called
void end_session(void* session) {
    printf("[DEBUG] Cleaning up buffer at %p\n", session);
    free(session);
}

// THE TARGET FUNCTION
void target_function() {
    void* current_session = create_session(128);
}

// MAIN EXECUTION (The Entry Point)
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: create_session
=====

>> MEMORY PROFILE:
{
  "return_type": "void*",
  "functional_description": "This function is a wrapper around `malloc`. It takes a single argument `size` (argument index 0), which specifies the number of bytes to allocate. The function allocates memory using `malloc(size)` and immediately returns the allocated memory. There are no conditional branches or additional logic in the function. The memory lifecycle is entirely managed by the caller of this function.",
  "tags": [
    "AllocSource<void*>::Ret"
  ]
}

=====
FUNCTION: end_session
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `end_session` is responsible for deallocating the memory pointed to by the incoming `session` pointer (argument index 0). It first prints a debug message with the address of the `session` pointer and then calls `free(session)` to release the memory. The function does not return any value, allocate new memory, or create aliases for the `session` pointer. The deallocation is unconditional, and there are no reference losses or passthroughs.",
  "tags": [
    "FreeSink<void*>::Arg{0}"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `target_function` calls the child function `create_session` with a fixed size of 128 bytes. The memory allocated by `create_session` is assigned to the local variable `current_session`. However, the function does not use, free, or pass out the allocated memory, resulting in a reference loss. The function has no incoming arguments, no conditional branches, and no outgoing parameters or return values.",
  "tags": [
    "InternalLeak<void*>::<current_session>"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> InternalLeak<void*>::<current_session>

>> 1 leak(s) require attention.
=====

```

Leak Logic Mapper Result

```

=====
==24086==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 128 byte(s) in 1 object(s) allocated from:
#0 0x7fd1d3ef4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x5632081571cb in create_session /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/02_missing_free_unsafe.c:18
#2 0x563208157215 in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/02_missing_free_unsafe.c:31
#3 0x56320815722a in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/02_missing_free_unsafe.c:36
#4 0x7fd1d3c35ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 128 byte(s) leaked in 1 allocation(s).

```

ASan Result

```

[DEBUG] Allocating 128 bytes...
==24128==
==24128== HEAP SUMMARY:
==24128==    in use at exit: 128 bytes in 1 blocks
==24128==    total heap usage: 2 allocs, 1 frees, 1,152 bytes allocated
==24128==
==24128== 128 bytes in 1 blocks are definitely lost in loss record 1 of 1
==24128==    at 0x4844818: malloc (vg_replace_malloc.c:446)
==24128==    by 0x10918B: create_session (02_missing_free_unsafe.c:18)
==24128==    by 0x1091D5: target_function (02_missing_free_unsafe.c:31)
==24128==    by 0x1091EA: main (02_missing_free_unsafe.c:36)
==24128==
==24128== LEAK SUMMARY:
==24128==    definitely lost: 128 bytes in 1 blocks
==24128==    indirectly lost: 0 bytes in 0 blocks
==24128==    possibly lost: 0 bytes in 0 blocks
==24128==    still reachable: 0 bytes in 0 blocks
==24128==    suppressed: 0 bytes in 0 blocks
==24128==
==24128== For lists of detected and suppressed errors, rerun with: -s
==24128== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

Analysis:

This is an exemplar test case demonstrating Leak Logic Mapper detecting an explicit missing free memory leak.

Figure 4-10: Missing Free Test Case

4.2.2 - Conditional Free

Code

```
/*
 * TEST CASE: This test houses a free statement within a conditional block.
 *
 * EXPECTED TAGS: [target_function()]: CondInternalLeak<int*>::<buffer>
 *
 * VULNERABILITY: Depending on the execution path, the memory may leak.
 */

// THE TARGET FUNCTION
void target_function(int cleanup_flag) {
    int* buffer = malloc(1024 * sizeof(int));
    if (buffer == NULL) return;
    printf("[DEBUG] Allocated 4096 bytes at %p...\n", (void*)buffer);
    if (cleanup_flag == 1) {
        free(buffer);
    }
    else {

    }
}

// MAIN EXECUTION
int main() {
    target_function(1);
    target_function(0);
    return 0;
}
```

```

=== DETAILED FUNCTION ANALYSIS ===

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates 4096 bytes of memory for an integer buffer using malloc and assigns it to the local variable 'buffer'. If the allocation fails, the function returns immediately. A debug message is printed to confirm the allocation. The function then checks the value of the 'cleanup_flag' argument. If 'cleanup_flag == 1', the allocated memory is freed using free(buffer). If 'cleanup_flag != 1', the function exits without freeing the memory, resulting in a memory leak. The function does not return the allocated memory or pass it to an out-parameter.",
  "tags": [
    "CondInternalLeak<int*>::<buffer>"
  ]
}

=====
FUNCTION: main
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The 'main' function calls 'target_function' twice with different arguments: first with '1' and then with '0'. The first call to 'target_function' results in memory being allocated and subsequently freed because the 'cleanup_flag' is '1'. The second call results in memory being allocated but not freed, leading to a memory leak because the 'cleanup_flag' is '0'. The 'main' function does not directly allocate, free, or pass through any memory. All memory management is encapsulated within 'target_function'.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> CondInternalLeak<int*>::<buffer>

>> 1 leak(s) require attention.

=====

```

Leak Logic Mapper Result (Passing Criteria)

```

=====
==13533==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 4096 byte(s) in 1 object(s) allocated from:
    #0 0x7f13870f4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x55cb3c9771ad in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/08_conditional_free_unsafe.c:14
    #2 0x55cb3c977202 in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/08_conditional_free_unsafe.c:33
    #3 0x7f1386e35ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 4096 byte(s) leaked in 1 allocation(s).

```

ASan Result

```

[DEBUG] Allocated 4096 bytes at 0x4a66040...
[DEBUG] Allocated 4096 bytes at 0x4a674c0...
==13572==
==13572== HEAP SUMMARY:
==13572==      in use at exit: 4,096 bytes in 1 blocks
==13572==    total heap usage: 3 allocs, 2 frees, 9,216 bytes allocated
==13572==
==13572== 4,096 bytes in 1 blocks are definitely lost in loss record 1 of 1
==13572==    at 0x4844818: malloc (vg_replace_malloc.c:446)
==13572==    by 0x10916D: target_function (08_conditional_free_unsafe.c:14)
==13572==    by 0x1091C2: main (08_conditional_free_unsafe.c:33)
==13572==
==13572== LEAK SUMMARY:
==13572==    definitely lost: 4,096 bytes in 1 blocks
==13572==    indirectly lost: 0 bytes in 0 blocks
==13572==    possibly lost: 0 bytes in 0 blocks
==13572==    still reachable: 0 bytes in 0 blocks
==13572==    suppressed: 0 bytes in 0 blocks
==13572==
==13572== For lists of detected and suppressed errors, rerun with: -s
==13572== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

Analysis:

This section details the observation and rectification of an identified fault.

```

>> LLM SEMANTIC PAYLOAD:
Sources: [
  {
    "allocation_statement": "buffer = malloc(1024 * sizeof(int));",
    "immediate_assignment": "none",
    "tracked_pointer": "orphaned_return",
    "variable_type": "int*",
    "alias_trace": [
      "orphaned_return"
    ],
    "is_reference_lost": true,
    "exit_alias": "none",
    "exit_portal": "none",
    "is_conditional": true
  },
  {
    "allocation_statement": "buffer = malloc(1024 * sizeof(int));",
    "immediate_assignment": "none",
    "tracked_pointer": "orphaned_return",
    "variable_type": "int*",
    "alias_trace": [
      "orphaned_return"
    ],
    "is_reference_lost": false,
    "exit_alias": "buffer",
    "exit_portal": "none",
    "is_conditional": false
  }
]
Sinks: [
  {
    "tracked_pointer": "buffer",
    "variable_type": "int*",
    "linked_allocation_statement": "buffer = malloc(1024 * sizeof(int));",
    "alias_trace": [
      "buffer"
    ],
    "free_statement": "free(buffer);",
    "is_conditional": false
  }
]

```

```
>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The 'main' function calls 'target_function' twice with different arguments.
  The first call, 'target_function(1)', allocates memory and then frees it, resulting in no memory leak.
  The second call, 'target_function(0)', allocates memory but does not free it, resulting in a memory leak.
  The 'main' function does not directly allocate, free, or pass memory. It relies entirely on the behavior of 'target_function'."
  "tags": [
    "CondInternalLeak<int*>::OrphanedReturn",
    "InternalLeak<int*>::OrphanedReturn"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
-> CondInternalLeak<int*>::<buffer>

[!] LEAK DETECTED: Function 'main'
-> CondInternalLeak<int*>::OrphanedReturn
-> InternalLeak<int*>::OrphanedReturn

>> 3 leak(s) require attention.
```

The LLM has incorrectly flagged two Orphaned Returns.

These errors occur when the LLM's attention mechanism over-weights the child's context. Which results in a hallucination where the LLM incorrectly extracts memory allocation statements belonging to the child function and attributes them to the parent function, leading to false-positive vulnerability tags. This can be termed as "Context Bleed". The system instruction was amended to directly compensate for this situation.

```
7. NEVER EXTRACT CHILD OPERATIONS: When building the Sources, Sinks, and ReferenceLosses arrays, you must only extract statements that are written inside the target function's lexical block. Do not hallucinate or extract malloc or free statements that occur inside a child function.
```

A new validating function was devised for the Orphaned Return cases. This validation function checks that the allocating statement can actually be found within the function's scope. If the allocation statement bled from the child function, the check fails and the tag is rejected.

```
def is_source_allocation_statement_in_code(llm_data: dict, validation_data: dict) -> bool:
    """
    Checks if the exact allocation statement claimed by the LLM actually exists within the target function's
    deterministic boundaries.
    """
    code = validation_data.get("func_code", "")

    statement = llm_data.get("allocation_statement", Schema.NONE)

    if statement == Schema.NONE: return False

    # Prevents the check from failing if the allocation statement includes the assignment
    if "=" in statement:
        statement = statement.split("=")[-1] # Takes everything after the last '='

    # Strip all whitespace to prevent LLM formatting hallucinations from failing the check
    normalised_code = "".join(code.split())
    normalised_statement = "".join(statement.split())

    return normalised_statement in normalised_code
```

Figure 4-11: Conditional Free Test Case

4.2.3 - Alloca

Code

```
/*
 * TEST CASE: This test case expects that the system will not recognise alloca as
 a heap allocation source.
 *
 * EXPECTED TAGS: There is no tag that encapsulates this behaviour in Leak Logic
 Mapper v1.0
 *
 * VULNERABILITY: The function allocates memory on the stack using alloca and
 returns the pointer. The memory is instantly corrupted upon return.
 */

// THE TARGET FUNCTION
// Dynamic allocation on the stack
int* target_function() {
    int* buffer = (int*)alloca(1024 * sizeof(int));
    printf("[DEBUG] Allocated buffer at %p...\n", (void*)buffer);
    buffer[0] = 42;
    return buffer;
}

// MAIN EXECUTION
// Accessing this is undefined behavior, the stack frame is gone.
int main() {
    int* ptr = target_function();
    printf("Accessing memory: %d\n", ptr[0]);
    return 0;
}
```

```
>> LLM SEMANTIC PAYLOAD:
Sources: []

Sinks: []

PassThroughs: []

ReferenceLosses: [
  {
    "tracked_pointer": "ptr",
    "variable_type": "int*",
    "alias_trace": [
      "ptr"
    ],
    "overwrite_statement": "int* ptr = target_function();",
    "overwritten_with": "invalid memory (stack-allocated in target_function)",
    "is_conditional": false
  }
]
```

```

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "int*",
  "functional_description": "The function allocates a buffer of 1024 integers on the stack using 'alloca'. It initializes the first element of the buffer to 42 and then returns a pointer to the buffer. However, since the memory is stack-allocated, it becomes invalid once the function returns, leading to undefined behavior. There are no conditional branches or child function calls affecting memory behavior.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
FUNCTION: main
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The function 'main' calls 'target_function', which returns a pointer to stack-allocated memory. The pointer is stored in the local variable 'ptr'. The function then attempts to access the memory pointed to by 'ptr' and prints its value. This results in undefined behavior because the memory is invalid after 'target_function' returns. The function does not perform any heap memory allocations, deallocations, or pointer passthroughs.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocated buffer at 0x7fff30f4cac0...
Accessing memory: 42

```

ASan Result

```

[DEBUG] Allocated buffer at 0x1ffeffe9f0...
==20408== Invalid read of size 4
==20408==    at 0x1091BC: main (10_alloca_safe.c:35)
==20408==   Address 0x1ffeffe9f0 is on thread 1's stack
==20408==  4144 bytes below stack pointer
==20408==
Accessing memory: 42
==20408==
==20408== HEAP SUMMARY:
==20408==   in use at exit: 0 bytes in 0 blocks
==20408==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==20408==
==20408== All heap blocks were freed -- no leaks are possible
==20408==
==20408== For lists of detected and suppressed errors, rerun with: -s
==20408== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

Analysis:

During initial development iterations, the `alloca` function was incorrectly populated as a source. Because `alloca` performs a stack-based allocation rather than a heap allocation, this test verifies that the system correctly distinguishes between the two and no longer misclassifies stack escapes as heap allocation sources.

As observed in the debug output, no valid heap source was populated. While the system's output of a `MemoryNeutral` tag is the expected behaviour that was tested for, it is misleading regarding overall program safety. A critical bug remains, caused by returning and accessing a dangling stack pointer. The LLM's semantic summary demonstrates that it is fully aware of the stack corruption that has

occurred. While implementing a dedicated StackEscape tag would resolve this, formally profiling vulnerabilities outside the strict scope of heap memory leaks is left for future work.

ASan disables stack use-after-return detection by default due to computational overhead. Because the required environment variable was not set during this test, ASan has not flagged the vulnerability. The integer 42 was still physically residing at the memory address 0x7fff30f4cac0; because no intervening code was executed to overwrite the popped stack frame before the pointer was accessed, the program silently read this. This is an example of how undefined behavior in C can deceptively create the impression that a flawed program is functioning as intended.

Figure 4-12: Alloca Test Case

4.2.4 - Conditional Leak

Code

```
/*
 * TEST CASE: This test case shows that the system should highlight a potential
 * flaw determining that the allocation may not on certain run time conditions.
 *
 * EXPECTED TAGS: [target_function()] CondInternalLeak<char*>::<data>
 *
 * VULNERABILITY: The allocation and free are dependent on random runtime return
 * conditions.
 */

// DEPENDENCY FUNCTIONS
int globalReturnsTrueOrFalse()
{
    return (rand() % 2);
}

// THE TARGET FUNCTION
void target_function() {
    char * data;
    data = NULL;
    data = (char *)malloc(100*sizeof(char));
    printf("[DEBUG] Allocated heap buffer at %p\n", data);
    if (data == NULL) {exit(-1);}
    strcpy(data, "A String");
    printf("%s\n", data);
    if(globalReturnsTrueOrFalse())
    {

    }
    else
    {
        printf("[DEBUG] Cleaning up buffer at %p\n", data);
        free(data);
    }
}

// MAIN EXECUTION
int main() {
    srand(time(NULL));
    target_function();
    return 0;
}
```



```

=====
FUNCTION: globalReturnsTrueOrFalse
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "This function generates a random integer using the `rand()` function, computes the remainder when divided by 2, and returns the result. The return value is either 0 or 1, determined non-deterministically by the output of `rand()`. The function does not allocate, free, or pass through any memory, nor does it overwrite any pointers.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "This function allocates 100 bytes of memory for a local pointer `data` and initializes it with a string. It then prints the string. The control flow diverges based on the non-deterministic return value of `globalReturnsTrueOrFalse()`. If the function returns true, the allocated memory is not freed, resulting in a reference loss. If it returns false, the memory is freed properly. The function does not return or pass the allocated memory to any out-parameter, and the memory lifecycle is entirely local to the function.",
  "tags": [
    "CondInternalLeak<char*>:::<data>"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> CondInternalLeak<char*>:::<data>

>> 1 leak(s) require attention.

=====

```

Leak Logic Mapper Result

```

=====
==8325==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 100 byte(s) in 1 object(s) allocated from:
    #0 0x7f558daf4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x55738003f240 in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/12_conditional_leak_unsafe.c:29
    #2 0x55738003f2f3 in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/12_conditional_leak_unsafe.c:51
    #3 0x7f558d835ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 100 byte(s) leaked in 1 allocation(s).
[DEBUG] Allocated heap buffer at 0x50b000000040
A String
[DEBUG] Cleaning up buffer at 0x50b000000040

```

ASan Results

```

[DEBUG] Allocated heap buffer at 0x4a66040
A String
==8379==
==8379== HEAP SUMMARY:
==8379==   in use at exit: 100 bytes in 1 blocks
==8379== total heap usage: 2 allocs, 1 frees, 1,124 bytes allocated
==8379==
==8379== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==8379==   at 0x4844818: malloc (vg_replace_malloc.c:446)
==8379==   by 0x1091E0: target function (12_conditional_leak_unsafe.c:29)
==8379==   by 0x109288: main (12_conditional_leak_unsafe.c:51)
==8379==
==8379== LEAK SUMMARY:
==8379==   definitely lost: 100 bytes in 1 blocks
==8379==   indirectly lost: 0 bytes in 0 blocks
==8379==   possibly lost: 0 bytes in 0 blocks
==8379==   still reachable: 0 bytes in 0 blocks
==8379==   suppressed: 0 bytes in 0 blocks
==8379==
==8379== For lists of detected and suppressed errors, rerun with: -s
==8379== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
==8380==
==8380== HEAP SUMMARY:
==8380==   in use at exit: 0 bytes in 0 blocks
==8380== total heap usage: 2 allocs, 2 frees, 1,124 bytes allocated
==8380==
==8380== All heap blocks were freed -- no leaks are possible
==8380==
==8380== For lists of detected and suppressed errors, rerun with: -s
==8380== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Valgrind Result

Analysis:

This scenario highlights a fundamental limitation of dynamic analysis. Because the memory leak is dependent on non-deterministic runtime conditions, tools such as ASan and Valgrind require multiple execution passes to hit the vulnerable path. Conversely, Leak Logic Mapper identifies the conditional vulnerability in a single pass by evaluating the overall code structure

Figure 4-13: Conditional Leak Test Case

5. Evaluation

This chapter evaluates the performance, reliability and architectural viability of Leak Logic Mapper, assessing the completed framework against the primary aim and objectives established in chapter 1. In addition to benchmarking the framework against industry-standard tools, this chapter synthesises observations from the development and testing phases to provide a comprehensive qualitative assessment.

5.1 Core Criteria:

5.1.1 Minimising Context Decay

It is well-established that utilising Large Language Models for large-scale tasks requiring extensive token inputs presents a significant challenge for context management. Feeding an LLM an excessive number of tokens simultaneously inevitably dilutes its attention mechanism. In traditional static analysis, a tool can traverse a substantial codebase without degrading its accuracy. Conversely, if an LLM is fed an entire C codebase it inevitably results in severe context decay and hallucination. As the token count increases, the model’s ability to retain variable states, accurately map pointers, and distinguish between disparate scopes heavily degrades. This results in “Context Bleed”, a form of hallucination where the LLM attributes logic or states from one function into another. Evaluating this criterion requires examining how effectively the system can analyse isolated code segments, without losing the broader program state.

5.1.2 Effective Abstraction of Heap Operations

There is a fundamental dichotomy between the nature of C programming and the nature of large language models: heap management has traditionally required mathematically rigid logic, whereas LLMs act as probabilistic text generators. While an LLM may excel at reliably parsing structural C syntax into standardised formats like JSON, there exists less experimentation in deducing strict memory logic. Abstraction is the key regarding the context decay dilemma; the most natural way to maximise attention is to reduce token overhead. The core challenge was abstracting a function to its essential heap data to enable context-sensitive static analysis, without introducing extraneous details that degrade accuracy. Evaluating this criterion necessitates examining the gap between semantic reasoning and the realities of heap operations.

Ultimately, assessing how effectively a system can translate probabilistic semantic extraction into a structured, reliable data format for downstream analysis.

5.1.3 Handling Inter-procedural Analysis

Memory vulnerabilities in C rarely exist in isolation; pointers are frequently allocated, aliased, and passed through multiple layers of function calls before eventually being freed or leaked. The challenge in inter-procedural analysis lies in accurately tracking a memory lifecycle across function boundaries where state representations shift between caller and callee scopes. For an LLM driven analyser, this mandates preservation of the transitional state of a pointer as it traverses the call graph. The evaluation of this criterion focuses on the complexity of propagating these states through nested architectures, and the systems ability to establish a contextual link between an analysis target and the abstracted information that a target depends on.

5.2 Evaluating LLM Comprehension

To evaluate theoretical criteria, validation requires investigating to what extent the underlying LLM can interpret and convey an understanding of diverse memory patterns. Memory allocations and deallocations rarely conform to a single uniform syntax. A robust static analyser must be capable of recognising standard boilerplate alongside complex anti-patterns.

Therefore, testing phases use unconventional aliases, conditional routes and wrappers for nested memory operations. It is necessary to determine whether the system genuinely understands the structural lifecycle of a pointer, rather than simply pattern matching explicit key words.

Analysing how accurately the LLM populates the JSON schema, specifically alias traces and exit portals, becomes a valuable measurement to the true extent of semantic reasoning. Compartmentalising where the LLM succeeds in structural abstraction, where it fails in probabilistic logic, and determining where the Python orchestrator has intervened facilitated clear analysis during development and retrospectively allows for critical reflection.

5.3 Static Analysis Comparison

Leak Logic Mapper is evaluated against two industry standard static analysis tools: Cppcheck (Marjamäki, 2024) and Clang Static Analyzer (The LLVM Project, 2026a). The comparison contextualises Leak Logic Mapper's performance by showcasing memory management scenarios where conventional tools struggle.

5.3.1 Cppcheck

Cppcheck is an open-source static analysis tool designed specifically for C and C++. Cppcheck's primary design philosophy is to achieve a minimal false-positive, ensuring that when a bug is flagged, it is highly likely to be a genuine issue. Cppcheck is designed to be able to analyse code with non-standard syntax, focussing on detecting undefined behaviour and dangerous coding constructs.

Cppcheck does not use a standard compiler frontend. Instead, it uses its own custom parser. The code is not required to be perfectly compilable or fully preprocessed, allowing the tool to parse incomplete code snippets; this shares a design philosophy with the tree-sitter-c approach in Leak Logic Mapper.

In Cppcheck, a program is preprocessed to an internal token list, which is analysed according to a list of predefined rules. Cppcheck is supplemented with value-flow and simple control-flow analysis. Users can customise checking rules to their needs.

```
int main()
{
    int*p = malloc(sizeof(int));

    *p = 5;

    printf("%d", *p);
}
```

```

8:1: error: Memory leak: p [memleak]

5:6: warning: If memory allocation fails, then there is a possible null pointer dereference: p [nullP
tOfMemory]
  5;

3:19: note: Assuming allocation function fails
p = malloc(sizeof(int));
  ^
3:19: note: Assignment 'p=malloc(sizeof(int))', assigned value is 0
p = malloc(sizeof(int));
  ^
5:6: note: Null pointer dereference
  5;

7:19: warning: If memory allocation fails, then there is a possible null pointer dereference: p [null
tOfMemory]
tf("%d", *p);
  ^
3:19: note: Assuming allocation function fails
p = malloc(sizeof(int));
  ^
3:19: note: Assignment 'p=malloc(sizeof(int))', assigned value is 0
p = malloc(sizeof(int));
  ^
7:19: note: Null pointer dereference
tf("%d", *p);
  ^

```

Figure 5-1: Cppcheck Intraprocedural Memory Leak

Cppcheck's scope for inter-procedural analysis is relatively limited. It tracks variable states and scopes primarily within the bounds of a single function. This adheres to the design goal producing minimal false positives, as aggressive inter-procedural tracking can lead to a large false positive rate. This limitation presents a specific challenge regarding wrapper functions; figure 5.2 showcases an example of an explicit memory leak occurring over two functions which has not been reported by Cppcheck:

```

int * allocator() {
    return malloc(sizeof(int));
}

int main()
{
    int*p = allocator();

    *p = 5;

    printf("%d", *p);
}

file1.c:1:0: style: The function 'allocator' should have static linkage since it is not used outside of its t
ranslation unit. [staticFunction]
int * allocator() {
^
nofile:0:0: information: Active checkers: 109/856 (use --checkers-report=<filename> to see details) [checkers
Report]

```

Figure 5-2: Interprocedural Memory Leak

The problem here is that while Cppcheck has built-in knowledge of how standard library functions like `malloc()` and `free()` behave, it cannot automatically assume that a custom wrapper function passes the ownership of dynamically allocated memory to the caller. Consequently, Cppcheck drops its ownership tracking to avoid false positives.

To make Cppcheck detect this leak, the analyser must explicitly be told that `allocator()` is a memory allocation function that requires deallocation. This is done using a custom XML configuration file:

```
<?xml version="1.0"?>
<def>
  <memory>
    <alloc>allocator</alloc>
    <dealloc>free</dealloc>
  </memory>
</def>
```

Figure 5-3: Cppcheck configuration file

When Cppcheck is explicitly told that `allocator` allocates memory and must be routed to a `free`, rerunning Cppcheck with the code in figure 5.2 now reports the leak:

```
file1.c:12:1: error: Memory leak: p [memleak]
}
^
```

Figure 5-4: Cppcheck Leak Acknowledgement

However, the example test case in Figure 5.5 demonstrates a further failure stemming from a context blindness issue, revealing how reliance on manual configuration can inadvertently mask deeper semantic vulnerabilities:

```
#include <stdlib.h>
#include <stdio.h>

int * allocator() {
    return malloc(sizeof(int));
}

void deallocator(int * p) {
    ;
}

int main()
{
    int*p = allocator();

    *p = 5;

    printf("%d", *p);

    deallocator(p);
}
```

```
<?xml version="1.0"?>
<def>
  <memory>
    <alloc>allocator</alloc>
    <dealloc>dealloc</dealloc>
  </memory>
</def>
```

```
Checking file1.c ...
```

Figure 5-5: Cppcheck False Negative

Cppcheck reported “*checking file1.c...*” meaning that no errors were found. Cppcheck does not evaluate the internal workings of `dealloc`, instead assuming that `dealloc` safely handles memory. Syntactically from the rules defined in the configuration file, this is correct, but without semantic analysis there is no verification of the actual error.

The vulnerability of this approach is that for the tool to accurately report a memory leak, the software tester must maintain an active, flawless understanding of all internal functions and their evolving memory contracts. If a function is refactored, deprecated or altered without a corresponding update to the decoupled configuration file, a static analysis test becomes desynchronised from the codebase.

While a manual inspection of the trivial program in Figure 5.5 readily reveals the empty `dealloc` function, relying on human verification negates the primary utility of automated static analysis. This practically becomes infeasible in large-scale codebases, encompassing thousands of lines of code, distributed teams, and deeply nested inter-procedural calls.

This silent failure is not a bug within Cppcheck’s parsing engine, but rather a direct consequence of its foundational design philosophy: the strict minimisation of false positives. When Cppcheck encounters a custom function that takes a pointer as an argument, it observes a ‘pointer escape’. Rather than pessimistically assuming the memory is leaked, the analyser makes an optimistic assumption that the function handles memory safely. Ultimately this paradigm generates a dangerous “illusion of safety”; the tool outputs a clean diagnostic report, instilling unwarranted confidence in the developer while critical memory vulnerabilities silently persist.

5.3.2 Clang Static Analyzer

The Clang Static Analyzer is a prominent, open-source analysis tool built on top of the LLVM compiler infrastructure (Lattner and Adve, 2004). While Cppcheck focuses on lightweight token matching and minimising false positives, Clang Static Analyzer takes a more mathematically rigorous approach. It relies on heavy path-sensitive symbolic execution to detect complex logical bugs, uninitialised values and memory management violations.

To perform its analysis, the tool leverages Clang (The LLVM Project, 2026b), a dedicated compiler front-end for the LLVM ecosystem. Unlike Leak Logic Mapper’s forgiving tree-sitter-c approach, the Clang front-end requires a fully compilable build environment to strictly parse the raw C code into a detailed AST. While a standard Clang compilation pipeline would normally pass this data directly down to generate machine code, the Static Analyzer instead intercepts it, translating the AST into a CFG.

Once the CFG is constructed, the analyser’s engine systematically explores the possible execution paths through this graph. As it traverses conditional branches and loops, it performs symbolic execution, placing constraints on variables to determine if a specific path is logically feasible.

To demonstrate the practical advantage of path-sensitive symbolic execution, Clang Static Analyzer was evaluated against the exact inter-procedural vulnerability that caused Cppcheck to fail silently (previously shown in figure 5.5).

When the pointer drops out of the scope at the end of *main()*, Clang’s *unix.Malloc* checker accurately intercepts the missing sink and successfully generates a memory leak warning. This is a core strength of mathematical simulation: by evaluating the explicit realities of the code, the tool eliminates the “illusion of safety” created by misconfigured manual rules.

```
scan-build: Using '/usr/lib/llvm-19/bin/clang' for static analysis
file1.c:21:1: warning: Potential leak of memory pointed to by 'p' [unix.Malloc]
    21 | }
       | ^
1 warning generated.
scan-build: Analysis run complete.
scan-build: 1 bug found.
scan-build: Run 'scan-view /tmp/scan-build-2026-05-05-133535-4702-1' to examine bug reports.
```

Figure 5-6: Clang Static Analyzer, Memory Leak

Clang Static Analyzer has been tested against Test Case 13 Reference Overwrite (Appendix C.13). In this scenario, a pointer is allocated in the target function and passed as a double

pointer into a child function, *buffer_reset()*. Within this child function, the dereferenced pointer is explicitly set to NULL before reassigning a new block of memory, irreversibly overwriting the original reference.

```

void buffer_reset(int** ptr_to_buffer) {
    *ptr_to_buffer = NULL;
    *ptr_to_buffer = (int*)malloc(2048 * sizeof(int));
    6 ← Potential memory leak →
}

// THE TARGET FUNCTION
void target_function() {
    int* buffer = (int*)malloc(1024 * sizeof(int));
    2 ← Memory is allocated →

    if (buffer == NULL) return;
    3 ← Assuming 'buffer' is not equal to NULL →

    4 ← Taking false branch →

    printf("[DEBUG] Allocated buffer at %p...\n", (void*)buffer);
    buffer[0] = 42;

    buffer_reset(&buffer);
    5 ← Calling 'buffer_reset' →

    if (buffer == NULL) return;

    printf("[INFO] Cleaning up buffer...\n");
    free(buffer);
}

// MAIN EXECUTION
int main() {
    target_function();
    1 ← Calling 'target_function' →

    return 0;
}

```

Figure 5-7: Clang Static Analyzer, Test Case 13

Clang Static Analyzer provides a visual debug report, *scan-view*, representing program control flow. Inside *buffer_reset()*, the engine has correctly recognised that the original memory address was no longer bound to any active variable in the simulated state, correctly flagging it as a leak. This achieves the exact same security outcome as Leak Logic Mapper, but through a fundamentally different architectural mechanism. Where Leak Logic Mapper relies on the LLM to semantically deduce and bubble up a ReferenceLoss as a formal warning for the parent function, Clang relies on rigorous state tracking across the function boundary.

Intra-procedural Nested Heap Structures:

Figure 5.7 illustrates a test case demonstrating a "shallow free" vulnerability. In this scenario, a nested heap structure (an array of pointers) is dynamically allocated and populated using a

for loop. At the end of the function, the program safely deallocates the outer array container but omits the necessary loop to free the individual inner elements. Consequently, the function exits with these nested allocations still residing on the heap, resulting in a multi-pointer memory leak.

```
void array_leak_test() {
    int num_elements = 5;

    // Allocate the outer array (an array of 5 integer pointers)
    int** num_array = (int**)malloc(num_elements * sizeof(int*));
    if (num_array == NULL) return;

    // Allocate memory for each individual integer inside the array
    for (int i = 0; i < num_elements; i++) {
        num_array[i] = (int*)malloc(sizeof(int));

        // ... function does some work with the array ...
        if (num_array[i] != NULL) {
            *num_array[i] = i * 10;
        }
    }

    printf("[DEBUG] Array populated successfully.\n");

    // VULNERABILITY: Shallow Free
    free(num_array);
}
```

Figure 5-8: Nested Heap Leak

When evaluated against this test case, the Leak Logic Mapper generated an accurate memory profile explicitly stating that the individual integer allocations were not freed. The LLM evaluated the code conceptually, it is immune to the computational constraints of loop unrolling. It natively recognises the “shallow free” anti-pattern, understanding that allocating elements within a loop requires a corresponding deallocation loop, without needing to manually track distinct memory regions for every iteration.

```

=====
FUNCTION: array_leak_test
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates an array of integer pointers (num_array) and then al-
locates memory for each individual integer in the array. It assigns values to the allocated integers. Th
e outer array (num_array) is freed at the end of the function, but the individual integer allocations (n
um_array[i]) are not freed, leading to memory leaks. The function does not return any value or pass poin
ters out.",
  "tags": [
    "InternalLeak<int*>::<num_array[i]>"
  ]
}

```

Figure 5-9: Leak Logic Mapper Nested Heap Leak

When evaluated against the same test case, Clang Static Analyzer failed to detect the memory leak, outputting a clean diagnostic report indicating no bugs were found. Its failure is rooted in computational resource management. Because simulating loops creates exponentially branching execution paths, a phenomenon known as state explosion, Clang Static Analyzer strictly bounds loop evaluations using internal thresholds. Once a loop exceeds this execution threshold, the engine loses precise symbolic tracking for individual array elements. Furthermore, the analyser uses conservative pointer escape heuristics. When the inner allocations are assigned to the dynamically allocated outer array, the analyser assumes that pointers have escaped into an opaque memory region and deliberately terminates tracking to prevent false positives (The LLVM Project, 2026b). Consequently, when the outer array is ultimately freed, the analyzer lacks the retained simulated state necessary to recognize that the inner pointers have been permanently orphaned.

```

scan-build -analyzer-config max-loop-iterations=10 gcc Nested_Heap_Leak.c -o Nested_Heap_Leak
scan-build: Using '/usr/lib/llvm-19/bin/clang' for static analysis
scan-build: Analysis run complete.
scan-build: Removing directory '/tmp/scan-build-2026-05-05-155111-10834-1' because it contains no report
s.
scan-build: No bugs found.

```

Figure 5-10: Clang Static Analyzer Nested Heap Leak

5.4 The Realloc Anti-Pattern

Throughout its evaluation, Leak Logic Mapper has shown a compelling ability to navigate complex memory states and understand genuine, nuanced vulnerabilities that are often mishandled by developers. A prime example of this is the system's evaluation of the realloc failure state.

In C programming, a common anti-pattern occurs when a developer assigns the result of `realloc` directly back to the original pointer. If the reallocation fails, it returns `NULL`. This instantly overwrites the reference to the original memory block meaning the initial allocation is orphaned before it can be routed to a free sink, resulting in a memory leak.

```
=====
FUNCTION: test_realloc_failure_state
=====

>> Code:
void test_realloc_failure_state() {

    int *buffer = (int *)malloc(10 * sizeof(int));
    if (buffer == NULL) {
        return;
    }

    buffer = (int *)realloc(buffer, 100 * sizeof(int));

    if (buffer == NULL) {
        printf("Memory reallocation failed!\n");

        return;
    }

    free(buffer);
}

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates memory for an integer buffer using malloc and attempts to resize it using realloc. If the initial malloc fails, the function returns immediately. If realloc fails, the function prints an error message and returns, causing a reference loss for the original memory allocated by malloc. If realloc succeeds, the resized memory is freed before the function exits. The function does not return or pass out any memory, and all control flow paths are deterministic.",
  "tags": [
    "CondInternalLeak<int*>::<buffer>"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'test_realloc_failure_state'
    -> CondInternalLeak<int*>::<buffer>

>> 1 leak(s) require attention.

=====
```

Figure 5-11: Realloc Leak

When Leak Logic Mapper evaluated this anti-pattern, the architecture successfully diagnosed the structural failure. It identifies that `malloc` and `realloc` act as distinct allocation sources, and the generated semantic summary demonstrates a clear understanding of the overwrite mechanism. Crucially, the system accurately captures the conditional nature of the bug. Because the original memory is safely freed during normal execution but orphaned specifically

on the path where `realloc` fails, the orchestrator prepends the conditional prefix. The resulting tag, `CondInternalLeak<int*>::<buffer>`, is precisely representative of this nuanced runtime vulnerability.

5.5 Opaque Boundaries and Unpredictable Variance.

A persistent challenge in static analysis is resolving opaque boundaries, particularly those created by dynamic dispatch mechanisms such as function pointers and callbacks. Because the specific function invoked by a pointer is only resolved at runtime, context-insensitive static analysers cannot definitively trace these execution paths at compile time.

Leak Logic Mapper is limited to unidirectional context, bottom-up, from child function calls. Dedicated handling for top-down context was out of scope during development of the system, however this presented a unique opportunity to evaluate how the system behaves when it is tasked to resolve a scenario without the necessary context. Analysis of the handling of these ambiguous states provides critical insight into the system’s baseline security posture and the non-deterministic nature of the underlying LLM.

To evaluate this, the system iteratively tested a *process_with_callback* function. This function allocates a heap buffer and passes it to a parameter typed as `CallbackFunc cb`. `Callback` was implemented as *typedef void (*CallbackFunc)(void*)*; however, because Leak Logic Mapper’s MVP scope excludes type-definition parsing, the AST extraction phase does not resolve this declaration. Therefore, the system is structurally blind to the programmatic implementation of this function pointer. Because the target function is isolated in this context-blind state, the implementation of the actual callback remains arbitrary. Consequently, it is structurally impossible for the analyser to definitively know the fate of the allocated memory.

```

=====
FUNCTION: process_with_callback
=====

>> Code:
void process_with_callback(CallbackFunc cb) {
    int *buffer = (int *)malloc(100 * sizeof(int));
    if (buffer == NULL) {
        return;
    }

    cb(buffer);
}

=====
FUNCTION: process_with_callback
=====

>> MEMORY PROFILE:
{
    "return_type": "void",
    "functional_description": "The function allocates memory for an integer buffer of size 100 and passes it to the provided callback function `cb`. If the allocation fails, the function returns immediately. The memory is not freed within this function, and ownership is assumed to be transferred to the callback.",
    "tags": [
        "AllocSource<int*>::Arg{0}",
        "PassThrough<int*>::Arg{None}ToArg{0}"
    ]
}

=====
FUNCTION: process_with_callback
=====

>> MEMORY PROFILE:
{
    "return_type": "void",
    "functional_description": "The function allocates memory for an integer buffer of size 100 and passes it to a callback function `cb`. If the allocation fails, the function returns immediately. The memory allocated for `buffer` is not freed within the function, and its lifecycle depends on the implementation of the callback function. Since the behavior of the callback is unknown, the reference to the allocated memory is considered lost.",
    "tags": [
        "InternalLeak<int*>::<buffer>"
    ]
}

>> MEMORY PROFILE:
{
    "return_type": "void",
    "functional_description": "The function `process_with_callback` allocates memory for 100 integers using `malloc` and assigns the address to the local variable `buffer` (argument index: none). It performs a NULL-check on `buffer` and returns immediately if the allocation fails. If the allocation succeeds, it calls the callback function `cb` with `buffer` as an argument. The function does not free the allocated memory or return it, resulting in a reference loss. The behavior of the callback function `cb` is unknown, so no assumptions are made about its memory operations.",
    "tags": [
        "InternalLeak<int*>::<buffer>"
    ]
}

```

Figure 5-12: Unsupported Callback Function Test Case

Across multiple evaluation runs, the LLM exhibited probabilistic variance in its semantic reasoning. In a minority of instances, the model attempted to logically resolve the boundary by hallucinating program intent. The model states, “ownership is assumed to be transferred to the

callback", the system further generated false AllocSource and PassThrough tags unrepresentative of the actual implementation of *process_with_callback*.

```
>> LLM SEMANTIC PAYLOAD:
Sources: [
  {
    "allocation_statement": "int *buffer = (int *)malloc(100 * sizeof(int));",
    "immediate_assignment": "local_variable",
    "tracked_pointer": "buffer",
    "variable_type": "int *",
    "alias_trace": [
      "buffer"
    ],
    "is_reference_lost": false,
    "exit_alias": "buffer",
    "exit_portal": "cb",
    "is_conditional": false
  }
]
```

Figure 5-13 Mispopulated Source Callback Function Test Case

This hallucinated schema population inadvertently exposed a secondary structural vulnerability within the system's validation layer. The LLM falsely designated the *cb* parameter as the *exit_portal* for the allocated source. Consequently, the Python orchestrator's *_evaluate_boundaries* function verified this claim against the AST parameters. Because the AST parser successfully identified *cb* as an incoming parameter, the validator approved the exit portal. The system's architectural blindness to typedef declarations meant it lacked the semantic context to differentiate between a data pointer (a valid out-parameter) and a function pointer (an invalid assignment target). This structural limitation allowed the LLM's hallucination to bypass the deterministic guardrails, resulting in the generation of false-negative safety tags.

This behavior highlights an inherent risk of utilising LLMs for security analysis: when starved of explicit structural context, their probabilistic nature can occasionally override strict system instructions, (see "*NO HEAP ASSUMPTIONS*" Appendix B, *SEMANTIC EXTRACTION RULES* 3.) to fill the contextual void. Furthermore, because these hallucinations stem from the non-deterministic nature of the underlying model, predicting, reproducing, and mitigating them presents a profound challenge for traditional software testing methodologies.

However, overall the system demonstrated a stronger tendency to lean toward conservative, safe analysis. In the majority of evaluation runs, the LLM exhibited analytical restraint and accurately identified the boundary's ambiguity. The model populated the JSON schema correctly, explicitly noting in its summaries that "*the behavior of the callback is unknown*" and "*no assumptions are made about its memory operations.*"

5.6 The Model Capability Problem

Empirical testing revealed a significant disparity in instruction adherence between enterprise-tier API models and locally hosted alternatives. Frontier models, such as GPT-4o, demonstrated a high capacity for structural fidelity, consistently parsing the prompt and generating valid JSON objects that strictly aligned with the requested data types and discrete schema domains.

Conversely, smaller local models, constrained by lower parameter counts and limited context windows, frequently exhibited instruction decay. When burdened with the heavy cognitive load of semantic C-code analysis, and complex formatting rules, hallucinated invalid JSON syntax or ignored strict schema constraints. A common failure state involved superimposing native constructs into the output, such as generating a null type instead of the schema mandated “none” string literal. The local inference pipeline was explicitly engineered with a defensive parsing mechanism to catch this hallucinated syntax, which would intercept and correct invalid null (None after translating a JSON payload to a python dictionary) types to the “none” string. However, if the model failed to generate a complete or structurally valid JSON object entirely, the deserialization step would inevitably fail, resulting in a pipeline crash.

Due to hardware constraints during development, local testing was restricted to models with 8 billion parameters or fewer. Consequently, determining whether this instruction decay is inherent to all local deployments or merely a limitation of small-scale parameters remains unresolved. Future work would necessitate scaling this analysis to larger open-weight models to better evaluate the balance between system efficacy and operational efficiency.

5.7 Evaluation Metrics

To rigorously evaluate the efficacy of Leak Logic Mapper, this study utilises the Juliet Test Suite, specifically targeting the CWE-401 (Missing Release of Memory after Effective Lifetime) vulnerability dataset. The framework's performance is quantified using standard classification metrics: Precision, Recall, and the F1 Score. To calculate these metrics, the tool's outputs must first be categorised into True Positives (correctly identified leaks), False Positives (incorrectly flagged safe code), and False Negatives (missed leaks).

Precision is then defined as the proportion of correctly identified vulnerabilities over all reported vulnerabilities, calculated as:

$$precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad (5.1)$$

Recall, measures the proportion of actual vulnerabilities that were correctly identified by the tool, given by:

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives} \quad (5.2)$$

Finally, to provide a comprehensive measure that balances both precision and recall, the F1 Score is calculated as their harmonic mean:

$$F1\ Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5.3)$$

To establish a controlled baseline for evaluation, the scope of the testing dataset was strictly limited to single-file C source code, deliberately excluding C++ variants and multi-file test cases. Within this defined scope, the Juliet Test Suite encapsulates multiple functions within a single test case file, typically yielding a ratio of one vulnerable function to four safe variants. The evaluation metrics for this study are calculated at the function level rather than the file level. This ratio provides a rigorously imbalanced dataset, aggressively testing the framework's resistance to False Positives.

To ensure the scientific validity of the evaluation phase, a strict pre-processing sanitisation pipeline was implemented to mitigate the risk of LLM data leakage. The Juliet Test Suite natively includes explicit vulnerability annotations (e.g., `/* POTENTIAL FLAW */`) and descriptive function typologies (e.g., `_bad()` or `good1()`). If left intact, a Large Language Model could bypass pure static analysis entirely, relying instead on textual sentiment and reading

comprehension to identify flaws. To enforce true topological and semantic reasoning, the evaluation harness applies a regular expression pre-processor prior to AST extraction. This pipeline systematically strips all block and inline comments and standardises function identifiers, neutralising vulnerable functions to `_target` and safe functions to `_variant`. By mathematically "blinding" the LLM to human-readable hints, the framework was forced to map pointer lifecycles and control flows strictly through code syntax, guaranteeing an unbiased measurement of its analytical capabilities.

The evaluation of the sanitized Juliet CWE-401 dataset encompassed 628 source files containing a total of 2,614 individual functions, yielding an overall F1 Score of 0.831. The framework recorded a Recall rate of 99.68%, correctly identifying 626 true vulnerabilities (True Positives) with only 2 missed leaks (False Negatives). Conversely, the Precision metric was measured at 71.30%. Out of the safe code evaluated, the system correctly categorized 1,734 functions with no leak (True Negative) but incorrectly flagged 252 safe functions as leaks (False Positive).

The near-zero False Negative rate is heavily influenced by the synthetic nature of the Juliet Test Suite. Designed fundamentally as a benchmark for traditional static analysis, the vulnerable functions are explicitly constructed to isolate and demonstrate a specific flaw. In many cases, a pointer is instantiated (e.g., via `malloc`), and the control flow deliberately terminates without any attempt at deallocation or complex pointer escaping. The LLM is presented with highly scoped, unambiguous C code. For a model of gpt-4o's capability, mapping an explicit source to a definitively absent sink is computationally straightforward for an advanced LLM.

Consequently, while the framework proved it does not "miss" explicit memory leaks, this 99.36% Recall rate must be understood within the context of synthetic benchmarking. In real-world, production-grade repositories (such as operating system kernels or complex distributed systems), memory leaks are rarely this explicit. They typically manifest through deeply nested concurrency failures, mismanaged struct arrays, or orphaned pointers within long-lived daemon loops. While Juliet proves the framework's baseline competence and fail-secure routing logic, future evaluations against real-world CVEs would be necessary to determine the False Negative rate in highly obfuscated, non-synthetic architectures.

See Appendix D for full test coverage.

5.8 Evaluation Synthesis

In summary, this evaluation demonstrates that Leak Logic Mapper fulfils its primary aim of architecting an LLM-driven inter-procedural analysis framework. By adhering to the core objectives, the system achieves a reduction in context decay through its modular, bottom-up pipeline that successfully abstracts heap operations into structured, validated memory profiles. The framework's ability to use memory profiles to resolve inter-procedural states across nested architectures ensures that pointer lifecycles remain traceable as they traverse disparate function scopes. The tool is fundamentally different to the traditional methods it has been compared to. Though the tool demonstrates aptitude resolving nuanced scenarios. There is a stochastic nature with LLMs and the inherent risk of hallucination. The validation engine and system instruction require continuous refinement during development and testing phases as unforeseen scenarios present unpredictable consequences. Computational resources available further raise complications, and where API calls can off load work at the cost of latency, this cordons a user base detecting vulnerabilities in a secure closed systems.

6. Conclusion

This chapter concludes the evaluation of the Leak Logic Mapper framework, reflecting on the project's achievements throughout the design, development and testing phases. It critically assesses the technical and architectural limitations of the current implementation and proposes a roadmap for future research in LLM-driven static analysis.

6.1 Completed Work

The primary outcome of this project is Leak Logic Mapper, an inter-procedural static analysis framework that bridges the semantic gap prevalent in traditional C memory management tools. Implemented in Python and utilising GPT-4o as its underlying inference engine, the framework represents a novel synthesis of deterministic parsing and semantic evaluation. Key milestones achieved during this project include:

- **Deterministic Parsing:** The integration of the tree-sitter-c library to enable reliable AST extraction.
- **Concurrent Orchestration:** The implementation of a thread-safe ThreadPoolExecutor to facilitate the parallel, bottom-up topological execution of the function call graph.
- **Structured Inference:** The design of a structured JSON schema that coerces the LLM into producing programmatically parsable, strictly typed memory analysis data.
- **Modular Validation Layer:** The development of a robust validation engine capable of cross-referencing LLM outputs against deterministic AST boundaries, effectively identifying and discarding contextual hallucinations.
- **Inter-Procedural Abstraction:** The reliable abstraction of complex target functions into succinct Memory Profiles, allowing pointer lifecycles to be tracked across function boundaries without inducing LLM context decay.
- **Test Suite Development:** The curation of a bespoke evaluation suite, inspired by the Juliet Test Suite, packaged with the system to rigorously demonstrate the framework's baseline proficiency.
- **Efficacy Benchmarking:** Comprehensive comparative testing against industry-standard static analysers, validating the framework's unique ability to detect semantic vulnerabilities that elude rigid symbolic execution engines.

6.2 Limitations

6.2.1 The Acyclic Constraint:

The primary architectural limitation of Leak Logic Mapper lies in its reliance on a DAG to define a call graph. By definition, a DAG cannot contain cycles, which fundamentally precludes the framework for resolving recursive function calls. Upon encountering cyclic dependencies, the orchestrator is unable to establish a deterministic evaluation order. Consequently, this creates a blind spot as the system cannot generate a finalised memory profile for a function that depends upon itself.

6.2.2 Scalability and Performance Overhead:

Traditional static analysers rely on mathematical models and symbolic execution, allowing them to traverse and evaluate codebases with rapid execution times. In contrast, the architecture of Leak Logic Mapper necessitates discrete LLM inference requests for every individual function node within the call graph. Although the implementation of a `ThreadPoolExecutor` facilitates concurrent analysis to partially mitigate this bottleneck, the framework remains fundamentally constrained by external network latency. Consequently, when applied to enterprise-scale C codebases, the compounding delays of API rate limits, synchronous network responses and cumulative token consumption incur prohibitive temporal and financial overhead. This currently limits the framework’s viability as a standalone tool for enterprise-scale code bases.

6.2.3: Edge Cases in C Semantics

The C programming language encompasses highly complex semantic features, such as function pointers, preprocessor macros and deep structural aliasing, that frequently obscure explicit inter-procedural control flow. In the current implementation, the AST parsing phase was deliberately constrained to a foundational level, as the primary research objective was to evaluate the efficacy of the LLM inference engine rather than to engineer a comprehensive C parser. However, reliably analysing advanced, real-world codebases necessitates that the framework accommodates these obscured inter-procedural connections. Consequently, this

limitation forces the project toward two diverging development paths for future iterations: either vastly increasing the architectural complexity of the deterministic parsing layer, or expanding the LLM's prompt architecture to infer dynamic routing.

6.2.4 LLM Non-Determinism and Prompt Fragility

Hallucinations were successfully using a modular validation layer and JSON coercion. However, despite strict schemas, LLMs are fundamentally probabilistic. Slight changes to the prompt or updates to the model weights could unexpectedly alter the inference quality. The validation layer does not adapt to non-determinism. If the validation layer and the LLM become out of sync, there may be times when the LLM has correctly inferred the state of memory operations within a function, but the validation layer does not accept this inference because the formatting was wrong.

6.2.5 Context Window vs Code Complexity

While the implementation of memory profile mitigates context decay by effectively abstracting child functions, the framework remains vulnerable to intra-procedural complexity. A single monolithic C function, characterised by high cyclomatic complexity, deep conditional nesting, or extensive line counts, could still overwhelm an LLM's context windows during its isolated analysis phase. When forced to process such a voluminous influx of raw tokens, the LLM's reasoning capabilities often deteriorate, resulting in token truncation, hallucinated pointer states, or a failure to accurately track variable life cycles across distant lines of code within the same function block. Consequently, while the abstraction layer effectively manages deep, nested call graphs, excessively large individual nodes still represent a significant barrier to inference quality.

6.3 Future Work

The following roadmap outlines key areas for future development, focussing on augmenting the deterministic preprocessing layer, expanding context resolution, and improving system scalability.

6.2.1 Path-Sensitive Conditional State Tracking

Currently, the framework’s handling of conditional memory operations is structurally path-insensitive. The conditional tag prefixes universally flag a probabilistic memory event but fail to quantify its explicit triggers; the tag represents a pointer freed on an isolated error-handling branch identically to one freed on the primary execution path. Future iterations should evolve the tagging system to correlate conditional memory events directly with the input values that trigger them. By extending the schema to support “When” conditions, for instance: `CondFreeSink<{type}>::Arg{idx}::When{Arg{k}==<state>}` the system can explicitly represent the exact pre-conditions required for a pointer lifecycle to terminate. When evaluating inter-procedural calls, orchestrating parent functions can dynamically evaluate their local arguments against these child guard constraints.

6.3.2 Advanced Preprocessing and Cross-Module Linking

Future iterations of the framework must substantially expand the deterministic parsing later to a robust preprocessing engine. The C compilation engine relies heavily on preprocessor transformations; thus, the framework must be capable of automatically expanding complex macros, resolving nested typedef structures, and evaluating conditional compilation directions (e.g., `#ifdef`) prior to AST generation. By statically resolving these constructs during the preprocessing phase, the system can prevent obscured pointer logic from reaching the LLM thereby reducing prompt fragility.

Furthermore, real-world software is rarely confined to a single translation unit. The orchestrator must be upgraded to support cross-module linking, allowing it to parse and correlate function declarations and definitions across multiple header and source files. Implementing a project-wide global call graph will enable the framework to track inter-procedural states regardless of file boundaries.

6.3.3 Hierarchical Tagging for Composite Data Structures

Currently, Leak Logic Mapper’s tagging system evaluates pointers as isolated, single-dimensional entities. However, real-world C applications heavily utilise composite data

structures, such as structs and multi-dimensional arrays, where memory allocations are nested within complex, multi-layered hierarchies. Future development could adapt the existing memory tags to track composite fields through deterministic access paths, for instance, evaluating a struct member as `AllocSource<struct -> member>` or an allocated array element as `FreeSink<matrix[0..N][0..M]>` (representing a symbolic range).

To facilitate this without overwhelming the LLM, the deterministic parsing layer must be expanded to extract explicit data structure definitions and member layouts directly from the source code. Dynamically injecting these targeted definitions as part of the LLM’s context payload could effectively ground the model’s probabilistic reasoning in the deterministic reality of the codebase. This contextual augmentation would empower the LLM to confidently assign standard tags to deeply nested memory states, effectively bridging the gap between flat structural parsing and complex memory hierarchies.

6.3.4 Resolving Indirect Calls via Top-Down Data-Flow Analysis

Future development must augment the framework with a preliminary top-down data-flow analysis phase. By forward-propagating function pointer assignments from their origin states, the parsing engine could statically deduce the potential target address of function pointers before the bottom-up LLM inference begins. Upon resolving these targets, the framework could employ context sensitive cloning, synthetically mutating the AST to substitute the indirect pointer with explicit function calls. In scenarios where a single pointer may dynamically route to multiple distinct functions, the orchestrator would dynamically generate variants of the calling function for each possible target. This allows the LLM to repeatedly evaluate the memory lifecycle across all potential execution contexts without fundamentally overhauling the bottom-up structure.

6.3.5 Resolving Cyclic Dependencies via Iterative Semantic Fixed-Points

To resolve recursive dependencies, future iterations of Leak Logic Mapper could implement a dynamic resolution phase utilising fixed-point iteration. To evaluate a cycle, a multi-pass analysis of the recursive node would be employed. In the initial “Base Case” pass, the LLM

would evaluate the function while treating recursive calls as empty operations, generating a foundational memory profile. In subsequent passes, the orchestrator would re-inject the profile from the previous iteration back into the LLM's context payload as the assumed state for the recursive call. This iterative loop continues until the LLM's output achieves semantic stability, the point at which the identified pointer lifecycles and memory tags remain identical across consecutive inferences. This state represents a "Fixed-Point," allowing the framework to mathematically and semantically bound the recursion. By locking in this stabilized profile, the system could resolve the cyclic dependency and resume its standard topological execution without violating the integrity of the broader call graph.

To support the advanced analytical capabilities of top-down data-flow and cyclic resolution, a fundamental refactoring of the framework's core orchestrator is required. The current implementation relies on a tightly coupled model where graph construction and functional evaluation occur in a nearly simultaneous, on-the-fly sequence. However, implementing path-sensitive heuristics and recursion handling necessitates a move toward decoupled, staged orchestration.

6.3.6 Two-Pass Architecture for Global State Resolution

Tracking out-of-band state modifiers, such as global and static variables, exposes a fundamental limitation of the bottom-up topological approach. Because the orchestrator evaluates leaf nodes prior to their callers, the analysis inherently lacks historical execution context. If a global variable is dynamically mutated by a parent function earlier in the runtime execution sequence, a purely bottom-up analysis will evaluate the child function using a stale or uninitialised global state.

Consequently, simply injecting global variable definitions into the LLM's context window is insufficient for accurate state tracking. To reliably resolve global memory mutations, future iterations of the Leak Logic Mapper might evolve into a Two-Pass Architecture. This would involve an initial Top-Down Data-Flow Pass to sequentially map global state mutations and propagate those expected states down the call graph. Only after this global context is

established would the framework initiate the Bottom-Up Semantic Pass, allowing the LLM to evaluate local pointer lifecycles against an accurate global state.

References

- Aslanyan, H. et al. (2022) “Static Analysis Methods For Memory Leak Detection: A Survey,” in *2022 Ivannikov Memorial Workshop (IVMEM)*, pp. 1–6. Available at: <https://doi.org/10.1109/IVMEM57067.2022.9983955>. (Accessed: 6 February 2026).
- Boland, T. and Black, P.E. (2012) “Juliet 1.1 C/C++ and Java Test Suite,” *Computer*, 45(10). Available at: <https://doi.org/10.1109/MC.2012.345>. (Accessed: 6 February 2026).
- Andersen, L. (1994) *Program Analysis and Specialization for the C Programming Language*. PhD thesis. University of Copenhagen. Available at: <https://www.cs.cornell.edu/courses/cs711/2005fa/papers/andersen-thesis94.pdf> (Accessed: 6 February 2026).
- Sui, Y. and Xue, J. (2016) ‘SVF: interprocedural static value-flow analysis in LLVM’, *Proceedings of the 25th International Conference on Compiler Construction*. New York: ACM, pp. 265–266. Available at: <https://dl.acm.org/doi/10.1145/2892208.2892235> (Accessed: 7 February 2026).
- Aldrich, J. (2013) ‘Pointer analysis’ [Lecture notes]. *15-8190: Program Analysis*. Carnegie Mellon University. Available at: <https://www.cs.cmu.edu/~aldrich/courses/15-8190-13sp/resources/pointer.pdf> (Accessed: 7 February 2026).
- Aslanyan, H. et al. (2024) ‘Combining static analysis with directed symbolic execution for scalable and accurate memory leak detection’, *IEEE Access*, 12, pp. 80128–80137. Available at: <https://doi.org/10.1109/ACCESS.2024.3409838> (Accessed: 7 February 2026).
- Hovhannisyan, H., Movsisyan, H. and Aslanyan, H. (2025) ‘Development and implementation of an algorithm for memory leak detection in C/C++ programs’, *CSIT Conference 2025*. Yerevan, Armenia, 22–26 September. Available at: https://csit.am/2025/proceedings/IC/IC_9.pdf (Accessed: 7 February 2026).
- Fang, C. et al. (2024) ‘Large language models for code analysis: do LLMs really do their job?’, in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA, 14-16 August. Available at: <https://www.usenix.org/conference/usenixsecurity24/presentation/fang> (Accessed: 8 May 2026).
- Gneciak, D. and Szandała, T. (2025) ‘Large language models versus static code analysis tools: a systematic benchmark for vulnerability detection’, *arXiv preprint arXiv:2508.04448*. Available at: <https://arxiv.org/abs/2508.04448> (Accessed: 8 May 2026).
- SonarSource (2026) *SonarQube* [Software]. Available at: <https://www.sonarsource.com/products/sonarqube/> (Accessed: 8 May 2026).
- GitHub (2026) *CodeQL* [Software]. Available at: <https://codeql.github.com/> (Accessed: 8 May 2026).
- Ahmed, M.B.U. et al. (2025) ‘SECVULEVAL: benchmarking LLMs for real-world C/C++ vulnerability detection’, *arXiv preprint arXiv:2505.19828*. Available at: <https://arxiv.org/abs/2505.19828> (Accessed: 8 May 2026).
- Huang, H., Shi, J., Wang, B., Yang, Z. and Lo, D. (2026) ‘Finding memory leaks in C/C++ programs via neuro-symbolic augmented static analysis’, *arXiv preprint arXiv:2603.27224*. Available at: <https://arxiv.org/abs/2603.27224> (Accessed: 8 May 2026).

Python Software Foundation (2024) *Python Language Reference, version 3.13* [Software]. Available at: <https://www.python.org> (Accessed: 17 April 2026).

Tree-sitter (2025) *tree-sitter-c* (Version 0.24.1) [Software]. Available at: <https://github.com/tree-sitter/tree-sitter-c> (Accessed: 17 March 2026).

OpenAI (2026) *JSON mode*. OpenAI. Available at: <https://platform.openai.com/docs/guides/text-generation/json-mode> (Accessed: 16 April 2026).

Rust Foundation (2024) *What is Ownership? - The Rust Programming Language*. Available at: <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html> (Accessed: 19 April 2026).

Wei, J. *et al.* (2022) 'Chain-of-thought prompting elicits reasoning in large language models', *Advances in Neural Information Processing Systems*, 35, pp. 24824–24837. Available at: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

Microsoft (2026) *Azure OpenAI Service models*. Available at: <https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/models> (Accessed: 30 April 2026).

OpenAI (2026) *API Reference*. Available at: <https://platform.openai.com/docs/api-reference> (Accessed: 30 April 2026).

Ollama (2026) *Ollama*. Available at: <https://ollama.com/> (Accessed: 30 April 2026).

Calcagno, C., *et al.* (2011) 'Compositional shape analysis by means of bi-abduction', *Journal of the ACM*, 58(6), pp. 1–66. Available at: <https://doi.org/10.1145/2049697.2049700>.

Kumar, S. (2026) *python-dotenv* [Software]. Available at: <https://pypi.org/project/python-dotenv/> (Accessed: 19 April 2026).

ISO/IEC (2011) *Information technology - Programming languages - C (Working Draft N1570)*. Available at: <https://www.open-std.org/jtc1/sc22/wg14/www/standards.html> (Accessed: 3 May 2026).

Nottingham, M. and Shapira, R. (2012) *Additional HTTP Status Codes*. RFC 6585. Available at: <https://www.rfc-editor.org/rfc/rfc6585> (Accessed: 30 April 2026).

Fielding, R., Nottingham, M. and Reschke, J. (2022) *HTTP Semantics*. RFC 9110. Available at: <https://www.rfc-editor.org/rfc/rfc9110> (Accessed: 30 April 2026).

Kahn, A.B. (1962) 'Topological sorting of large networks', *Communications of the ACM*, 5(11), pp. 558–562. Available at: <https://doi.org/10.1145/368996.369025>.

Serebryany, K., Bruening, D., Potapenko, A. and Vyukov, D. (2012) 'AddressSanitizer: a fast address sanity checker', *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. Boston, MA, 13–15 June. Berkeley: USENIX Association, pp. 309–318. Available at: <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany> (Accessed: 3 May 2026).

Nethercote, N. and Seward, J. (2007) 'Valgrind: a framework for heavyweight dynamic binary instrumentation', *Proceedings of the 28th ACM SIGPLAN Conference on*

Programming Language Design and Implementation (PLDI '07). San Diego, California, USA, 10-13 June. New York: ACM, pp. 89–100. Available at: <https://doi.org/10.1145/1250734.1250746> (Accessed: 3 May 2026).

Free Software Foundation (2024) *GNU Compiler Collection*. Version 14.2 [Software]. Available at: <https://gcc.gnu.org/> (Accessed: 3 May 2026).

Marjamäki, D. (2026) *Cppcheck* (Version 2.20.0) [Software]. Available at: <https://cppcheck.sourceforge.io/> (Accessed: 4 May 2026).

The LLVM Project (2026a) *Clang Static Analyzer* [Software]. Available at: <https://clang-analyzer.llvm.org/> (Accessed: 5 May 2026).

The LLVM Project (2026b) *Clang: a C language family frontend for LLVM* [Software]. Available at: <https://clang.llvm.org/> (Accessed: 5 May 2026).

Lattner, C. and Adve, V. (2004) 'LLVM: a compilation framework for lifelong program analysis & transformation', in *International Symposium on Code Generation and Optimization, 2004. CGO 2004*. Palo Alto, CA, USA, 20-24 March. IEEE, pp. 75–86. Available at: <https://doi.org/10.1109/CGO.2004.1281665>.

Appendices

Appendix A - System Instruction:

You are a deterministic Static Memory Analyzer for C code. Analyze the provided 'Target Function' and output a single valid JSON object matching this exact schema:

```
{
  "analysis_scratchpad": "string (Step-by-step reasoning. First, explicitly map all local variables to their incoming argument indices. Explicitly state if conditions are deterministic based on Child Function profiles.)",
  "function_name": "string",
  "functional_description": "A summary of the function's memory lifecycle. If it is a wrapper, state what it returns. Detail all control flow logic. FORMATTING RULE: Whenever you mention a local variable that is an alias of an incoming parameter, you must append its argument index in parentheses.",
  "sources": [
    {
      "allocation_statement": "string (The exact line of C code for allocation)",
      "immediate_assignment": "string (The left-hand side of the allocation_statement. Strictly: \"local_variable\", \"return_statement\", \"out_parameter\", or \"none\")",
      "tracked_pointer": "string (The initial variable name holding the memory)",
      "variable_type": "string (The C data type)",
      "alias_trace": ["string (Chronological chain of aliases tracking this specific address)"],
      "is_reference_lost": "boolean (True ONLY if the address becomes unreachable without being freed or passed out)",
      "exit_alias": "string (The final variable name holding the memory, or \"none\" if the reference was lost)",
      "exit_portal": "string (Strictly: \"return_statement\", the literal name of the destination argument for out_parameters, or \"none\")",
      "is_conditional": "boolean (True if this allocation/exit path is skip-able)"
    }
  ],
  "sinks": [
    {
      "tracked_pointer": "string (The name of the pointer/parameter at function entry, or \"none\" for local sources)",
      "variable_type": "string (The C data type)",
      "linked_allocation_statement": "string (Matches 'allocation_statement' if freeing a local source, else \"none\")",
      "alias_trace": ["string (Chain of aliases leading to the free)"],
      "free_statement": "string (The exact line of C code)",
      "is_conditional": "boolean (True if this deallocation path is skip-able)"
    }
  ],
  "passthroughs": [
    {
      "tracked_pointer": "string (The parameter name at entry. Include pointers involved in cross-assignments or swaps)",
      "variable_type": "string (The C data type)",
      "alias_trace": ["string (Chain of aliases tracking the movement of the address)"],
      "exit_portal": "string (Strictly: \"return_statement\" or the destination argument name)",
      "exit_statement": "string (The exact line of C code where the pointer is passed out)",
      "is_conditional": "boolean (True if the transfer of ownership is skip-able)"
    }
  ]
}
```

```

"reference_losses": [
  {
    "tracked_pointer": "string (The name of the pointer being overwritten. Do
not log cross-assignments or swaps here; log them in passthroughs)",
    "variable_type": "string (The C data type)",
    "alias_trace": ["string ()"],
    "overwrite_statement": "string (The line where the reference is lost)",
    "overwritten_with": "string (e.g., \"NULL\", \"0\", or a new address)",
    "is_conditional": "boolean (True if the overwrite is skip-able)"
  }
]
}

```

DEFINITIONS & PRIORITY RULES:

1. TRACKED POINTER: The primary variable name associated with a memory address at the start of an operation.
2. ESCAPING / ALIASING (PassThrough): If a pointer is assigned to an exit_portal (Return or Out-Parameter), the memory address has successfully escaped the function. This merely creates a copy of the address (an alias). It is not a destructive move. The original variable still holds the valid address unless explicitly overwritten. Swaps and cross-assignments are valid escaping operations.
3. REFERENCE LOSS: This strictly models "Orphaned Memory." If a pointer is overwritten (e.g., ptr = NULL) after it has been passed to an exit_portal or freed, it is not a Reference Loss. Only log a Reference Loss if the address becomes irrecoverable.
4. ALIAS TRACING: You must track the address. If `*out = *p`, then `*out` is now an alias for the address in `*p`.

CHILD FUNCTION TAG RULES:

1. "AllocSource": Treat as a new allocation (Source).
2. "FreeSink": Treat as a deallocation (Sink).
3. "PassThrough": Ownership is preserved and transferred. Do not log a Reference Loss for pointers passed into a child with this tag.
4. "ReferenceLoss": Identity is destroyed. If a pointer enters a child with this tag and no PassThrough tag is present for that same pointer, set `is_reference_lost`: true.

INTER-PROCEDURAL CONTROL FLOW RULES:

If a memory operation occurs inside a conditional block, you must analyze the condition using the "Child Function Memory Profiles".

1. DETERMINISTIC EVALUATION: Read the `functional_description` of any child function evaluated in the condition. If the description indicates the function's return value or behavior is strictly deterministic (e.g., always returning a specific constant, always returning NULL, or always failing), you must resolve the condition statically.
2. GUARANTEED EXECUTION: If the deterministic evaluation proves the condition will always be met, treat the block as the main execution path. Set all memory operations within it as `is_conditional`: false.
3. DEAD CODE: If the deterministic evaluation proves the condition will never be met, the block cannot execute. Do not log any sources, sinks, or passthroughs contained within it.
4. EXIT PORTALS: An exit portal is strictly a pathway out of the CURRENTLY ANALYZED Target Function (e.g., returning from the function, or assigning to an argument of the Target Function). Local variables that merely receive data from child functions are local aliases, not exit portals.

SEMANTIC EXTRACTION RULES

1. Ignore standard NULL-guards (e.g., `if (!ptr) return;`) when determining `is_conditional` for both newly allocated memory and incoming arguments. Always assume the memory allocation succeeds and treat the successful path as unconditional, do not set `is_conditional`: true for a standard NULL-guard.
2. Never consolidate divergent control flow paths into a single object. If a pointer has a success path (e.g., exiting via return) and a failure path (e.g., returning NULL and leaking), you must output two distinct objects in the sources array.

3. NO HEAP ASSUMPTIONS: Because you evaluate functions in isolation, you will often see pointers entering as arguments without knowing their origin. You must assume all incoming pointers represent trackable memory. Do not ignore pointer movements, swaps, or assignments just because you cannot see an explicit allocation in the current function's scope.

4. IGNORED RETURNS: If a function call known to return allocated memory (an "AllocSource") is not assigned to any variable or is not immediately returned, you must still log it in the Sources array. Set "tracked_pointer": "orphaned_return" and "immediate_assignment": "none".

5. INVALID SINKS: If a pointer is passed into a free() or a child function with a "FreeSink" tag, but you determine the variable no longer holds the valid memory address (e.g., it was previously swapped, reassigned, or nullified), do not link it to the original allocation. Set "linked_allocation_statement" to "none".

6. MUTUALLY EXCLUSIVE ARRAYS: A PassThrough STRICTLY models the movement of an INCOMING argument to an outgoing boundary. Locally allocated memory belongs ONLY in the Sources array. If a newly allocated pointer escapes the function (e.g., assigned to an out-parameter or returned), map that strictly using the "exit_portal" field in the Sources array. Never log a local Source in the PassThroughs array.

7. NEVER EXTRACT CHILD OPERATIONS: When building the Sources, Sinks, and ReferenceLosses arrays, you must only extract statements that are written inside the target function's lexical block. Do not hallucinate or extract malloc or free statements that occur inside a child function.

C-LANGUAGE SEMANTICS:

1. C-ALIASING: In C, passing a pointer to an out-parameter or assigning it to another variable merely copies the memory address (aliasing). It does not invalidate or consume the original pointer. The original pointer can still be safely used or passed to free(). Do not mark a reference as lost or unlink a Sink just because the pointer was aliased.

Generate your intermediate thinking steps before producing the final JSON. Put these steps in the analysis_scratchpad field.

Appendix B - Validator Functions:

```
# Verify internal control flow
def require_allocator(llm_data: dict, validation_data: dict) -> bool:
    """
    Discard if the allocating function in the allocation statement
    has not already been validated.
    """
    statement = llm_data.get("allocation_statement", Schema.NONE)
    if statement == Schema.NONE: return False
    return any(alloc in statement for alloc in
validation_data.get("known_allocators"))

def require_deallocator(llm_data: dict, validation_data: dict) -> bool:
    """
    Discard if the deallocating function in the freeing statement
    has not already been validated.
    """
    statement = llm_data.get("free_statement", Schema.NONE)
    if statement == Schema.NONE: return False
    return any(dealloc in statement for dealloc in
validation_data.get("known_deallocators"))

def exit_alias_in_returns(llm_data: dict, validation_data: dict) -> bool:
    """
    Checks if the exit_alias exists in the AST return map.
    """
    alias = llm_data.get("exit_alias", Schema.NONE)
    if alias == Schema.NONE: return False
    norm_alias = normalise_return_statements(alias)
```

```

    return any(norm_alias in ret for ret in validation_data["rets_map"])

def is_immediate_return(llm_data: dict, validation_data: dict) -> bool:
    """
    Applies a bi-directional check for if the normalised allocation matches an AST
    return.
    """
    statement = llm_data.get("allocation_statement", Schema.NONE)
    if statement == Schema.NONE: return False

    norm_statement = normalise_return_statements(str(statement))
    return any((norm_statement in ret) or (ret in norm_statement) for ret in
validation_data["rets_map"])

def is_source_allocation_statement_in_code(llm_data: dict, validation_data: dict) -
> bool:
    """
    Checks if the exact allocation statement claimed by the LLM actually exists
    within the target function's deterministic boundaries.
    """
    code = validation_data.get("func_code", "")

    statement = llm_data.get("allocation_statement", Schema.NONE)

    if statement == Schema.NONE: return False

    # Prevents the check from failing if the allocation statement includes the
assignment
    if "=" in statement:
        statement = statement.split("=")[-1] # Takes everything after the last '='

    # Strip all whitespace to prevent LLM formatting hallucinations from failing the
check
    normalised_code = "".join(code.split())
    normalised_statement = "".join(statement.split())

    return normalised_statement in normalised_code

def entry_arg_is_not_equal_to_exit_portal(llm_data: dict, validation_data: dict) ->
bool:
    """
    Checks that a pointer is not passed back to the same argument in which it
    entered.
    """
    tracked_pointer = llm_data.get("tracked_pointer", Schema.NONE)
    exit_portal = llm_data.get("exit_portal", Schema.NONE)

    if tracked_pointer == Schema.NONE or exit_portal == Schema.NONE: return False

    _, clean_entry = normalise_parameters(str(tracked_pointer))
    _, clean_exit = normalise_parameters(str(exit_portal))
    return clean_entry != clean_exit

def is_not_freed(llm_data: dict, validation_data: dict) -> bool:
    """
    Checks if an allocated pointer lacks a guaranteed free.
    """
    allocation_statement = llm_data.get("allocation_statement", Schema.NONE)
    if allocation_statement == Schema.NONE: return True

    # it is a guaranteed leak. Even if a free() exists later, it's freeing
NULL/garbage.
    if llm_data.get("is_reference_lost", False):
        return True

    # Check if there is an associating sink for a locally allocated source
    sinks = validation_data.get("sinks", [])

```

```

    linked_sinks = [sink for sink in sinks if
sink.get("linked_allocation_statement") == allocation_statement]
    if not linked_sinks: return True

    # If one sink is unconditional, the memory is safely handled on the main path.
    has_unconditional_sink = any(not sink.get("is_conditional", False) for sink in
linked_sinks)
    if has_unconditional_sink: return False

    # Conditional Leak.
    return True

def is_mutating_argument(llm_data: dict, validation_data: dict) -> bool:
    """
    Validates that the pointer being acted upon was passed in as an argument, rather
    than being a purely internal local variable.
    """
    raw_entry = str(llm_data.get("tracked_pointer", Schema.NONE))
    if raw_entry == Schema.NONE: return False

    _, clean_entry = normalise_parameters(str(raw_entry))
    return clean_entry in validation_data.get("param_map", {})

```

Appendix C - Extended Test Suite Results:

Each table in this section follows the following hierarchical format: Test Case Identifier -> Test Case Code -> Leak Logic Mapper Output -> Asan Output -> Valgrind Output -> Additional notes (optional)

C.01 - Basic Allocation

Code

```
/*
 * TEST CASE: This test demonstrates how the system handles basic allocation/deallocation
 * wrapper functions.
 *
 * EXPECTED TAGS:   [source_buffer()] AllocSource<void*>::Ret
 *                  [cleanup_buffer()] FreeSink<void*>Arg{0}
 *                  [target_function()] MemoryNeutral
 *
 * VULNERABILITY: No vulnerabilities.
 */

// DEPENDENCY FUNCTIONS
// Safe AllocSource
void* source_buffer(size_t size) {
    printf("[DEBUG] Allocating %zu bytes...\n", size);
    void * p = malloc(size);
    return p;
}

// Safe FreeSink
void cleanup_buffer(void* ptr) {
    printf("[DEBUG] Cleaning up buffer at %p\n", ptr);
    free(ptr);
}

// TARGET FUNCTION
void target_function() {
    void* my_data = source_buffer(1024);
    if (my_data == NULL) {
        return;
    }
    cleanup_buffer(my_data);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: cleanup_buffer
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `cleanup_buffer` is responsible for deallocating memory pointed to by the incoming argument `ptr` (argument index 0). It prints a debug message indicating the address being freed and then calls `free(ptr)` to release the memory. The function does not allocate new memory, return any values, or pass out pointers. The deallocation is unconditional and deterministic.",
  "tags": [
    "FreeSink<void*>::Arg{0}"
  ]
}

=====
FUNCTION: source_buffer
=====

>> MEMORY PROFILE:
{
  "return_type": "void*",
  "functional_description": "The function `source_buffer` allocates a block of memory of the specified size using `malloc` and returns a pointer to the allocated memory. The memory is not freed within the function, and it is the caller's responsibility to manage the memory lifecycle. The function is deterministic and always attempts to allocate memory and return the result of `malloc`. If `malloc` fails, it will return `NULL`, which is also returned by this function.",
  "tags": [
    "AllocSource<void*>::Ret"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `target_function` manages a memory allocation and deallocation lifecycle. It first calls `source_buffer(1024)` to allocate a block of memory and assigns the result to the local variable `my_data`. If the allocation fails (i.e., `my_data` is `NULL`), the function returns immediately without performing any further operations. If the allocation succeeds, the function calls `cleanup_buffer(my_data)` to free the allocated memory. The function ensures that all allocated memory is freed before it exits, and there are no reference losses or passthroughs. The control flow is deterministic, with a single conditional check for `NULL` after the allocation.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocating 1024 bytes...
[DEBUG] Cleaning up buffer at 0x519000000580

```

ASan Result

```

==23596==
==23596== HEAP SUMMARY:
==23596==    in use at exit: 0 bytes in 0 blocks
==23596==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==23596==
==23596== All heap blocks were freed -- no leaks are possible
==23596==
==23596== For lists of detected and suppressed errors, rerun with: -s
==23596== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.03 - Pointer Swap

Code

```
*
* TEST CASE: This test case demonstrates a function that takes two pointers and swaps them.
*
* EXPECTED TAGS: [simple_swap()] PassThrough<int**>::Arg{0}ToArg{1}
*               PassThrough<int**>::Arg{1}ToArg{0}
*
* VULNERABILITY: No Vulnerabilities
*/

// DEPENDENCY FUNCTIONS
// Swaps the memory addresses held by two pointers.
void simple_swap(int** a, int** b) {
    int* temp = *a;
    *a = *b;
    *b = temp;
}

// THE TARGET FUNCTION
void target_function() {
    int* x = malloc(sizeof(int));
    *x = 100;
    printf("[DEBUG] Allocating %zu bytes at %p [x = %d]\n", sizeof(int), x, *x);
    int* y = malloc(sizeof(int));
    *y = 200;
    printf("[DEBUG] Allocating %zu bytes at %p [y = %d]\n", sizeof(int), y, *y);
    if (x == NULL || y == NULL) return;
    simple_swap(&x, &y);
    printf("[DEBUG] Cleaning up buffer at %p [x = %d]\n", x, *x);
    free(x);
    printf("[DEBUG] Cleaning up buffer at %p [y = %d]\n", y, *y);
    free(y);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: simple_swap
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `simple_swap` swaps the memory addresses stored in the double pointers `a` (argument index 0) and `b` (argument index 1). It uses a local variable `temp` to temporarily hold the value of `*a` while performing the swap. The function does not allocate or free memory, and the control flow is deterministic. The memory addresses in `*a` and `*b` are preserved and swapped, effectively passing through the memory addresses between the two arguments.",
  "tags": [
    "PassThrough<int*>::Arg{0}ToArg{1}",
    "PassThrough<int*>::Arg{1}ToArg{0}"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `target_function` allocates two integer-sized memory blocks using `malloc` and assigns their addresses to local variables `x` and `y`. It initializes the allocated memory with the values 100 and 200, respectively. The function then calls `simple_swap`, which swaps the memory addresses stored in `x` and `y`. After the swap, the memory pointed to by `x` and `y` is freed using `free(x)` and `free(y)`. The function includes a NULL-guard to handle allocation failures, but this path is not considered in the main execution flow. The control flow is deterministic, and all memory is properly freed before the function exits.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
FUNCTION: main
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The `main` function serves as an entry point for the program. It calls `target_function`, which is responsible for allocating, swapping, and freeing memory. The `main` function does not directly allocate, free, or pass through any memory. After calling `target_function`, it returns 0, indicating successful execution.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

```

=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `simple_swap` swaps the integer pointers stored in the memory locations pointed to by `a` (argument index 0) and `b` (argument index 1). It uses a local variable `temp` to temporarily hold the value of `*a` during the swap. The function does not allocate or deallocate memory, and the control flow is deterministic. The function modifies the values of `*a` and `*b`, effectively passing through the pointers they hold.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocating 4 bytes at 0x502000000010 [x = 100]
[DEBUG] Allocating 4 bytes at 0x502000000030 [y = 200]
[DEBUG] Cleaning up buffer at 0x502000000030 [x = 200]
[DEBUG] Cleaning up buffer at 0x502000000010 [y = 100]

```

ASan Result

```
==24932==  
==24932== HEAP SUMMARY:  
==24932==    in use at exit: 0 bytes in 0 blocks  
==24932==   total heap usage: 3 allocs, 3 frees, 1,032 bytes allocated  
==24932==  
==24932== All heap blocks were freed -- no leaks are possible  
==24932==  
==24932== For lists of detected and suppressed errors, rerun with: -s  
==24932== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Valgrind Result

Additional Notes:

This test case highlights complexities in the model's schema population, specifically resulting in the intermittent omission of PassThrough tags. Despite this inconsistent attribution, the detail provided in the functional description provides sufficient context to ensure the final classification remains accurately memory neutral.

C.04 - PassThrough Return

Code

```
/*
 * TEST CASE: This test demonstrates that the system can keep track of a pointer that is
 * passed through and safely utilised then aliased by a child function.
 *
 * EXPECTED TAGS: [activate_session()] PassThrough<SessionData*>::Arg{0}ToRet
 *               [target_function()] MemoryNeutral
 *
 * VULNERABILITY: No vulnerabilities.
 */

// DATA STRUCTURES
struct SessionData {
    int session_id;
    int is_active;
};

// DEPENDENCY FUNCTIONS
// Safe PassThrough
struct SessionData* activate_session(struct SessionData* session) {
    if (session != NULL) {
        session->is_active = 1;
    }
    return session;
}

// THE TARGET FUNCTION
void target_function() {
    struct SessionData* my_session = malloc(sizeof(struct SessionData));
    printf("[DEBUG] Allocating %zu bytes at %p...\n", sizeof(struct SessionData),
my_session);
    struct SessionData* active_session = activate_session(my_session);
    printf("[DEBUG] Cleaning up buffer at %p\n", my_session);
    free(active_session);
}

// MAIN EXECUTION (The Entry Point)
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: activate_session
=====

>> MEMORY PROFILE:
{
  "return_type": "struct SessionData*",
  "functional_description": "The function takes a pointer to a `struct SessionData` as input. If the pointer is not NULL, it sets the `is_active` field of the structure to 1. The function then returns the same pointer. This function does not allocate or free memory, and it acts as a passthrough for the incoming pointer.",
  "tags": [
    "PassThrough<structSessionData*>::Arg{0}ToRet"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates memory for a `struct SessionData` object and assigns it to the local variable `my_session`. It then calls the `activate_session` function, which returns the same pointer, assigning it to the local variable `active_session`. Finally, the function frees the memory using `free(active_session)`. The memory lifecycle is fully managed within the function, with no memory escaping or being leaked. The control flow is straightforward and deterministic, with no conditional branches.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocating 8 bytes at 0x502000000010...
[DEBUG] Cleaning up buffer at 0x502000000010

```

ASan Result

```

==7524==
==7524== HEAP SUMMARY:
==7524==   in use at exit: 0 bytes in 0 blocks
==7524==   total heap usage: 2 allocs, 2 frees, 1,032 bytes allocated
==7524==
==7524== All heap blocks were freed -- no leaks are possible
==7524==
==7524== For lists of detected and suppressed errors, rerun with: -s
==7524== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Valgrind Result

Additional Notes

As observed in the generated PassThrough tag, the variable type `struct SessionData*` is rendered without whitespace as `structSessionData*`. This is not an extraction error by the LLM, but rather a side effect of the normalisation step enforced by the system's profiling module. Profiler.py aggressively strips whitespace from type declarations in an attempt to ensure consistent, deterministic tag formatting. As the core information is still present in the tag, this is acceptable.

C.05 - PassThrough Out Parameter

Code

```
/*
 * TEST CASE: This test demonstrates that the system can keep track of a pointer that is
 * aliased via out parameters.
 *
 * EXPECTED TAGS: [transfer_session_reference()] PassThrough<SessionData*>::Arg{0}ToArg{1}
 *               [target_function()] MemoryNeutral
 *
 * VULNERABILITY: No vulnerabilities.
 */

// DATA STRUCTURES
struct SessionData {
    int session_id;
    int is_active;
};

// DEPENDENCY FUNCTIONS
// Safe Passthrough
void transfer_session_reference(struct SessionData* src, struct SessionData** dest) {
    if (src != NULL && dest != NULL) {
        *dest = src;
    }
}

// THE TARGET FUNCTION
void target_function() {
    struct SessionData* original_session = malloc(sizeof(struct SessionData));
    printf("[DEBUG] Allocating %zu bytes at %p...\n", sizeof(struct SessionData),
original_session);
    struct SessionData* retrieved_session = NULL;
    transfer_session_reference(original_session, &retrieved_session);
    printf("[DEBUG] Cleaning up buffer at %p\n", retrieved_session);
    free(retrieved_session);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: transfer_session_reference
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function transfers the reference of a `struct SessionData` pointer (`src`) to an out-parameter (`dest`). If both `src` and `dest` are non-NULL, it assigns the value of `src` to the dereferenced `dest`, creating an alias for the memory address held by `src`. The function does not allocate or deallocate memory, nor does it overwrite any references. It is a simple passthrough operation for the `src` pointer.",
  "tags": [
    "CondPassThrough<structSessionData*>:Arg{0}ToArg{1}"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates memory for a `struct SessionData` object and assigns the pointer to the local variable `original_session`. It then calls `transfer_session_reference`, which transfers the reference of `original_session` to the local variable `retrieved_session`. Finally, the function frees the memory pointed to by `retrieved_session`. All operations are deterministic and executed unconditionally. The function ensures proper cleanup of the allocated memory, with no reference losses or leaks.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocating 8 bytes at 0x502000000010...
[DEBUG] Cleaning up buffer at 0x502000000010

```

ASan Result

```

==8097==
==8097== HEAP SUMMARY:
==8097==   in use at exit: 0 bytes in 0 blocks
==8097== total heap usage: 2 allocs, 2 frees, 1,032 bytes allocated
==8097==
==8097== All heap blocks were freed -- no leaks are possible
==8097==
==8097== For lists of detected and suppressed errors, rerun with: -s
==8097== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Valgrind Result

Additional Notes

As observed in the output for *transfer_session_reference*, the system generated a *CondPassThrough* tag. Although the semantic extraction rules explicitly instruct the Large Language Model to disregard standard NULL guards when evaluating conditionality, the model has interpreted the *if* statement as a divergent execution path. However, this prefix does not negatively impact the wider inter-procedural analysis. The system gracefully accepts the tag; in the parent function it is explicit that unless *malloc* fails, both *src** and *dest*** would be non-NULL, correctly resulting in a *MemoryNeutral* profile for the parent *target_function*.

C.06 - Orphaned Return

Code

```
/*
 * TEST CASE: This case tests an orphaned return, the return pointer from the allocating
 * call process is not stored.
 *
 * EXPECTED TAGS: [process()] AllocSource<Node*>::Ret
 *               [target_function()] InternalLeak<Node*>::OrphanedReturn
 *
 * VULNERABILITY: Orphaned return.
 */

// DATA STRUCTURES
struct Node {
    int data;
};

// DEPENDENCY FUNCTIONS
// AllocSource
struct Node* process(struct Node* incoming) {
    struct Node* a = incoming;
    a = malloc(sizeof(struct Node));
    return a;
}

// THE TARGET FUNCTION
// Call process but do not store the returned pointer.
void target_function() {
    struct Node* myNode = malloc(sizeof(struct Node));
    printf("[DEBUG] Allocating %zu bytes at %p...\n", sizeof(struct Node), myNode);
    process(myNode);
    printf("[DEBUG] Cleaning up buffer at %p\n", myNode);
    free(myNode);
}

// MAIN EXECUTION (The Entry Point)
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: process
=====

>> MEMORY PROFILE:
{
  "return_type": "struct Node*",
  "functional description": "The function 'process' takes a pointer to a 'struct Node' as input. It declares a local variable 'a', initially aliasing it to the input pointer 'incoming(0)'. However, 'a' is reassigned to point to newly allocated memory for a 'struct Node' using malloc. The function then returns this newly allocated memory. The original 'incoming' pointer is not used after the alias is broken. The function deterministically allocates memory and returns it, with no conditional logic or reference loss."
,
  "tags": [
    "AllocSource<structNode*>::Ret"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional description": "The function 'target_function' allocates memory for a 'struct Node' and assigns it to the local variable 'myNode'. It then calls the function 'process', passing 'myNode' as an argument. The 'process' function deterministically allocates new memory and returns it, but this returned memory is not captured or used in 'target_function', resulting in an orphaned allocation. After the call to 'process', 'target_function' frees the memory originally allocated to 'myNode'. All memory operations are deterministic, and there is no conditional logic affecting the control flow.",
  "tags": [
    "InternalLeak<structNode*>::OrphanedReturn"
  ]
}

```

```

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> InternalLeak<structNode*>::OrphanedReturn

>> 1 leak(s) require attention.

=====

```

Leak Logic Mapper Result

```

=====
==12077==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 4 byte(s) in 1 object(s) allocated from:
    #0 0x7f70964f4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x564e44e55186 in process /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/06_orphaned_return_unsafe.c:19
    #2 0x564e44e551b2 in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/06_orphaned_return_unsafe.c:28
    #3 0x564e44e551cf in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/06_orphaned_return_unsafe.c:36
    #4 0x7f7096235ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 4 byte(s) leaked in 1 allocation(s).

```

ASan Result

```

[DEBUG] Allocating 4 bytes at 0x4a66040...
[DEBUG] Cleaning up buffer at 0x4a66040
==12239==
==12239== HEAP SUMMARY:
==12239==     in use at exit: 4 bytes in 1 blocks
==12239==   total heap usage: 3 allocs, 2 frees, 1,032 bytes allocated
==12239==
==12239== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12239==    at 0x4844818: malloc (vg_replace_malloc.c:446)
==12239==    by 0x109176: process (06_orphaned_return_unsafe.c:20)
==12239==    by 0x1091C2: target_function (06_orphaned_return_unsafe.c:30)
==12239==    by 0x1091FA: main (06_orphaned_return_unsafe.c:39)
==12239==
==12239== LEAK SUMMARY:
==12239==    definitely lost: 4 bytes in 1 blocks
==12239==    indirectly lost: 0 bytes in 0 blocks
==12239==    possibly lost: 0 bytes in 0 blocks
==12239==    still reachable: 0 bytes in 0 blocks
==12239==    suppressed: 0 bytes in 0 blocks
==12239==
==12239== For lists of detected and suppressed errors, rerun with: -s
==12239== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.07 - Overwritten Return

Code

```
/*
 * TEST CASE: This test case stores the returned pointer from process but overwrites myNode
in the act of doing so.
 *
 * EXPECTED TAGS: [process()] AllocSource<Node*>::Ret
 *                [target_function()] InternalLeak<Node*>::<myNode>
 *
 * VULNERABILITY: Overwritten pointer in target_function()
 */

#include <stdlib.h>
#include <stdio.h>

// DATA STRUCTURES
struct Node {
    int data;
};

// DEPENDENCY FUNCTIONS
struct Node* process(struct Node* incoming) {
    struct Node* a = incoming;
    a = malloc(sizeof(struct Node));
    return a;
}

// THE TARGET FUNCTION
// Overwrite our original pointer with the returned pointer.
void target_function() {
    struct Node* myNode = malloc(sizeof(struct Node));
    printf("[DEBUG] Allocating %zu bytes at %p...\n", sizeof(struct Node), myNode);
    myNode = process(myNode);
    printf("[DEBUG] Cleaning up buffer at %p\n", myNode);
    free(myNode);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```



```

=====
FUNCTION: process
=====

>> MEMORY PROFILE:
{
  "return_type": "struct Node*",
  "functional_description": "The function 'process' takes a pointer to a struct Node as input. It declares a local variable 'a', initially aliasing the input pointer 'incoming(0)'. However, 'a' is reassigned to a newly allocated memory block for a struct Node using malloc. The function then returns this newly allocated memory. The function does not free any memory or perform any other operations. The control flow is linear and deterministic.",
  "tags": [
    "AllocSource<structNode*>::Ret"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function 'target_function' allocates memory for a struct Node and assigns it to the local variable 'myNode'. It then calls the function 'process', passing 'myNode' as an argument. The function 'process' allocates a new block of memory and returns it, which is then assigned back to 'myNode', overwriting the original memory address and causing a reference loss for the initial allocation. Finally, the function frees the memory currently held by 'myNode' (the one returned by 'process'). The control flow is linear and deterministic.",
  "tags": [
    "InternalLeak<structNode*>::<myNode>"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> InternalLeak<structNode*>::<myNode>

>> 1 leak(s) require attention.
=====

```

Leak Logic Mapper Result

```

=====
==12805==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 4 byte(s) in 1 object(s) allocated from:
    #0 0x7fb653cf4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x5605f8afd1d2 in target_function /home/j/uni/synaptic_project/Leak_Logic Mapper/tests/sample_test_cases/07_overwritten_pointer_unsafe.c:27
    #2 0x5605f8afd23e in main /home/j/uni/synaptic_project/Leak_Logic Mapper/tests/sample_test_cases/07_overwritten_pointer_unsafe.c:39
    #3 0x7fb653a35ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 4 byte(s) leaked in 1 allocation(s).

```

ASan Result

```

[DEBUG] Allocating 4 bytes at 0x4a66040...
[DEBUG] Cleaning up buffer at 0x4a664d0
==12900==
==12900== HEAP SUMMARY:
==12900==      in use at exit: 4 bytes in 1 blocks
==12900==    total heap usage: 3 allocs, 2 frees, 1,032 bytes allocated
==12900==
==12900== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12900==    at 0x4844818: malloc (vg_replace_malloc.c:446)
==12900==    by 0x109192: target_function (07_overwritten_pointer_unsafe.c:27)
==12900==    by 0x1091FE: main (07_overwritten_pointer_unsafe.c:39)
==12900==
==12900== LEAK SUMMARY:
==12900==    definitely lost: 4 bytes in 1 blocks
==12900==    indirectly lost: 0 bytes in 0 blocks
==12900==    possibly lost: 0 bytes in 0 blocks
==12900==    still reachable: 0 bytes in 0 blocks
==12900==    suppressed: 0 bytes in 0 blocks
==12900==
==12900== For lists of detected and suppressed errors, rerun with: -s
==12900== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.09 - Early Return

Code

```
/*
 * TEST CASE: This test houses an early return within a conditional block, exiting the
function without performing proper cleanup.
 *
 * EXPECTED TAGS: [target_function()]: CondInternalLeak<int*>::<buffer>
 *
 * VULNERABILITY: The function allocates memory but returns early on an error path without
freeing the buffer.
 */

// THE TARGET FUNCTION
void target_function(int error_flag) {
    int* buffer = malloc(1024 * sizeof(int));
    if (buffer == NULL) return;
    printf("[DEBUG] Allocated buffer at %p\n", (void*)buffer);
    if (error_flag == 1) {
        return;
    }
    free(buffer);
}

// MAIN EXECUTION
int main() {
    target_function(0);
    target_function(1);
    return 0;
}
```

```

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function allocates a buffer of integers using malloc and assigns it to the local variable 'buffer'. If the allocation fails (buffer == NULL), the function returns immediately. A debug message is printed with the address of the allocated buffer. The function then checks the value of 'error_flag' (argument index 0). If 'error_flag' is 1, the function returns without freeing the buffer, causing a memory leak. Otherwise, the buffer is freed before the function exits. The control flow is deterministic based on the value of 'error_flag'.",
  "tags": [
    "CondInternalLeak<int*>::<buffer>"
  ]
}

=====
FUNCTION: main
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "The 'main' function serves as an entry point and calls 'target_function' twice with different arguments (0 and 1). It does not perform any memory allocation, deallocation, or pointer manipulation itself. The memory lifecycle is entirely managed within 'target_function', which is invoked with deterministic control flow based on the value of its 'error_flag' argument. The first call to 'target_function' (with argument 0) results in proper memory management, as the buffer is freed before the function exits. The second call (with argument 1) causes a memory leak, as the buffer is not freed. The 'main' function itself does not directly interact with memory and simply returns 0 after the calls.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
VULNERABILITY SUMMARY
=====

[!] LEAK DETECTED: Function 'target_function'
    -> CondInternalLeak<int*>::<buffer>

>> 1 leak(s) require attention.

=====

```

Leak Logic Mapper Result

```

=====
==14554==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 4096 byte(s) in 1 object(s) allocated from:
    #0 0x7f8a5eaf4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x5581074f81bd in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/09_early_return_unsafe.c:15
    #2 0x5581074f8233 in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/09_early_return_unsafe.c:33
    #3 0x7f8a5e835ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 4096 byte(s) leaked in 1 allocation(s).
=====

```

ASan Result

```

[DEBUG] Allocated buffer at 0x4a66040
[DEBUG] Allocated buffer at 0x4a674c0
==14645==
==14645== HEAP SUMMARY:
==14645==     in use at exit: 4,096 bytes in 1 blocks
==14645==   total heap usage: 3 allocs, 2 frees, 9,216 bytes allocated
==14645==
==14645== 4,096 bytes in 1 blocks are definitely lost in loss record 1 of 1
==14645==    at 0x4844818: malloc (vg_replace_malloc.c:446)
==14645==    by 0x10916D: target function (09_early_return_unsafe.c:15)
==14645==    by 0x1091C5: main (09_early_return_unsafe.c:31)
==14645==
==14645== LEAK SUMMARY:
==14645==    definitely lost: 4,096 bytes in 1 blocks
==14645==    indirectly lost: 0 bytes in 0 blocks
==14645==    possibly lost: 0 bytes in 0 blocks
==14645==    still reachable: 0 bytes in 0 blocks
==14645==    suppressed: 0 bytes in 0 blocks
==14645==
==14645== For lists of detected and suppressed errors, rerun with: -s
==14645== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.11 - Deterministic Control Flow

Code

```
/*
 * TEST CASE: This case tests the systems ability to use contextual clues to understand
 control flow.
 *
 * EXPECTED TAGS: [target_function()] MemoryNeutral
 *
 * VULNERABILITY: No vulnerabilities.
 */

// DEPENDENCY FUNCTIONS
static int staticReturnsTrue()
{
    return 1;
}

static int staticReturnsFalse()
{
    return 0;
}

// THE TARGET FUNCTION
void target_function() {
    char * data;
    data = NULL;
    if(staticReturnsTrue())
    {
        data = (char *)malloc(100*sizeof(char));
        if (data == NULL) {exit(-1);}

        printf("[DEBUG] Allocated buffer at %p\n", data);

        strcpy(data, "A String");
        printf("%s\n", data);
    }
    if(staticReturnsFalse())
    {
    }
    else
    {
        printf("[DEBUG] Cleaning up buffer at %p\n", data);
        free(data);
    }
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: staticReturnsFalse
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "This function deterministically returns the constant value 0. It does not allocate, deallocate, or manipulate memory in any way. The control flow is straightforward with no branches or conditions.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
FUNCTION: staticReturnsTrue
=====

>> MEMORY PROFILE:
{
  "return_type": "int",
  "functional_description": "This function always returns the constant value 1. It does not perform any memory allocation, deallocation, or pointer manipulation. The return value is deterministic and can be used to resolve conditions in other functions.",
  "tags": [
    "MemoryNeutral"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "This function allocates a buffer of 100 characters if `staticReturnsTrue()` returns 1 (which it always does). It initializes the buffer with the string `\"A String\"` and prints it. If `staticReturnsFalse()` returns 0 (which it always does), the function frees the allocated buffer. All control flow paths are deterministic, and the memory lifecycle is fully managed within the function. There are no reference losses or passthroughs.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[DEBUG] Allocated buffer at 0x50b000000040
A String
[DEBUG] Cleaning up buffer at 0x50b000000040

```

ASan Result

```

==5118==
==5118== HEAP SUMMARY:
==5118==   in use at exit: 0 bytes in 0 blocks
==5118==   total heap usage: 2 allocs, 2 frees, 1,124 bytes allocated
==5118==
==5118== All heap blocks were freed -- no leaks are possible
==5118==
==5118== For lists of detected and suppressed errors, rerun with: -s
==5118== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.13 - Reference Overwrite

Code

```
/*
 * TEST CASE: This test takes a pointer to a pointer and overwrites it with a new memory
 * allocation, causing the original reference to be overwritten and lost.
 *
 * EXPECTED TAGS: [child_function()]: ReferenceLoss<int*>::Arg{0}
 *                AllocSource<int*>::Arg{0}
 *                [target_function]: InternalLeak<int*>::<buffer>
 *
 * VULNERABILITY: There is a missing free in the buffer_reset function, the reference to the
 * original buffer is lost without being freed when it is set to NULL, causing a memory leak.
 */

// DEPENDENCY FUNCTIONS
// Unsafe AllocSource
void buffer_reset(int** ptr_to_buffer) {
    *ptr_to_buffer = NULL;
    *ptr_to_buffer = (int*)malloc(2048 * sizeof(int));
}

// THE TARGET FUNCTION
void target_function() {
    int* buffer = (int*)malloc(1024 * sizeof(int));
    if (buffer == NULL) return;

    printf("[DEBUG] Allocated buffer at %p...\n", (void*)buffer);
    buffer[0] = 42;

    buffer_resest(&buffer);

    if (buffer == NULL) return;

    printf("[INFO] Cleaning up buffer...\n");
    free(buffer);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```



```

=====
FUNCTION: buffer_reset
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `buffer_reset` resets the memory address stored in the pointer `*ptr_to_buffer`. First, it overwrites the existing address in `*ptr_to_buffer` with `NULL`, potentially causing a reference loss if the previous address was valid and not freed. Then, it allocates a new memory block of size `2048 * sizeof(int)` and assigns the resulting pointer to `*ptr_to_buffer`. The function does not free any memory or pass any pointers out of its scope.",
  "tags": [
    "AllocSource<int*>::Arg{0}",
    "ReferenceLoss<int*>::Arg{0}"
  ]
}

=====
FUNCTION: target_function
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `target_function` manages a dynamically allocated integer buffer. It first allocates memory for an integer array of size 1024 and assigns the pointer to the local variable `buffer`. If the allocation fails, the function exits early. The function then calls `buffer_reset`, which overwrites the existing address in `buffer` with NULL (causing a reference loss) and assigns a new allocation of size 2048 to `buffer`. After the call to `buffer_reset`, the function checks if `buffer` is NULL and exits early if it is. If `buffer` is not NULL, the function frees the memory before exiting. The function ensures that no memory is leaked from the final state, but the original allocation is leaked due to the behavior of `buffer_reset`.",
  "tags": [
    "InternalLeak<int*>::<buffer>"
  ]
}

```

Leak Logic Mapper Result

```

=====
==8930==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 4096 byte(s) in 1 object(s) allocated from:
    #0 0x7f64eb0f4c57 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
    #1 0x561367b962b7 in target_function /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/13_reference_overwrite_unsafe.c:26
    #2 0x561367b96450 in main /home/j/uni/synaptic_project/Leak_Logic_Mapper/tests/sample_test_cases/13_reference_overwrite_unsafe.c:42
    #3 0x7f64eae35ca7 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58

SUMMARY: AddressSanitizer: 4096 byte(s) leaked in 1 allocation(s).

```

ASan Result

```

[DEBUG] Allocated buffer at 0x4a66040...
[INFO] Cleaning up buffer...
==8944==
==8944== HEAP SUMMARY:
==8944==   in use at exit: 4,096 bytes in 1 blocks
==8944== total heap usage: 3 allocs, 2 frees, 13,312 bytes allocated
==8944==
==8944== 4,096 bytes in 1 blocks are definitely lost in loss record 1 of 1
==8944==   at 0x4844818: malloc (vg_replace_malloc.c:446)
==8944==   by 0x1091A8: target_function (13_reference_overwrite_unsafe.c:26)
==8944==   by 0x109220: main (13_reference_overwrite_unsafe.c:42)
==8944==
==8944== LEAK SUMMARY:
==8944==   definitely lost: 4,096 bytes in 1 blocks
==8944==   indirectly lost: 0 bytes in 0 blocks
==8944==   possibly lost: 0 bytes in 0 blocks
==8944==   still reachable: 0 bytes in 0 blocks
==8944==   suppressed: 0 bytes in 0 blocks
==8944==
==8944== For lists of detected and suppressed errors, rerun with: -s
==8944== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

```

Valgrind Result

C.14 - Deep Nesting

Code

```
/*
 * TEST CASE: This test case demonstrates that the will bubble up tags for wrapper functions
 *
 * EXPECTED TAGS: [layer_3_cleanup()] FreeSink<int*>::<data>
 *                [layer_2_pass()] FreeSink<int*>::<data>
 *                [layer_1_pass()] FreeSink<int*>::<data>
 *                [target_function()] MemoryNeutral
 *
 * VULNERABILITY: No Vulnerabilities.
 */

// DEPENDENCY FUNCTIONS
// Layer 3 The actual FreeSink
void layer_3_cleanup(int* data) {
    printf("[INFO] Layer 3: Cleaning up buffer at %p...\n", (void*)data);
    free(data);
}

// FreeSink Wrapper
void layer_2_pass(int* data) {
    printf("[DEBUG] Layer 2: Passing pointer down...\n");
    layer_3_cleanup(data);
}

// FreeSink Wrapper
void layer_1_pass(int* data) {
    printf("[DEBUG] Layer 1: Passing pointer down...\n");
    layer_2_pass(data);
}

// TARGET FUNCTION
void target_function() {
    int* buffer = (int*)malloc(1024 * sizeof(int));
    if (buffer == NULL) return;
    printf("[DEBUG] Allocated buffer at %p...\n", (void*)buffer);
    buffer[0] = 42;
    layer_1_pass(buffer);
}

// MAIN EXECUTION
int main() {
    target_function();
    return 0;
}
```

```

=====
FUNCTION: layer_4_cleanup
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `layer_4_cleanup` is responsible for deallocating the memory pointed to by the incoming argument `data` (int*). It logs a message indicating the address being cleaned up and then calls `free(data)` to release the memory. The function does not return any value, and the deallocation is unconditional. There are no local allocations, passthroughs, or reference losses in this function.",
  "tags": [
    "FreeSink<int*>::Arg{0}"
  ]
}

=====
FUNCTION: layer_3_pass
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `layer_3_pass` is a wrapper that passes the incoming pointer `data` (int*) to the child function `layer_4_cleanup`. The child function unconditionally deallocates the memory pointed to by `data` using `free(data)`. The function does not perform any other operations or return a value. The control flow is straightforward and deterministic, with no conditional branches.",
  "tags": [
    "FreeSink<int*>::Arg{0}"
  ]
}

=====
FUNCTION: layer_2_pass
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `layer_2_pass` is a simple wrapper that takes an integer pointer `data` as input and passes it to the child function `layer_3_pass`. The child function `layer_3_pass` further passes the pointer to `layer_4_cleanup`, which unconditionally frees the memory pointed to by `data`. The function does not perform any other operations or return a value. The control flow is straightforward and deterministic, with no conditional branches.",
  "tags": [
    "FreeSink<int*>::Arg{0}"
  ]
}

=====
FUNCTION: layer_1_allocate
=====

>> MEMORY PROFILE:
{
  "return_type": "void",
  "functional_description": "The function `layer_1_allocate` allocates memory for an array of 1024 integers using `malloc` and assigns the pointer to the local variable `buffer`. It checks if the allocation was successful using a standard NULL-guard and returns immediately if the allocation fails. If the allocation succeeds, it initializes the first element of the array to 42 and passes the pointer `buffer` to the child function `layer_2_pass`. According to the child function memory profile, the memory is unconditionally freed in a downstream function (`layer_4_cleanup`). The function does not return any value or perform any other operations.",
  "tags": [
    "MemoryNeutral"
  ]
}

```

Leak Logic Mapper Result

```

[INFO] Starting Deep Nesting Execution...
[DEBUG] Layer 1: Allocated buffer at 0x521000000100...
[DEBUG] Layer 2: Passing pointer down...
[DEBUG] Layer 3: Passing pointer down...
[INFO] Layer 4: Cleaning up buffer at 0x521000000100...

```

ASan Result

```
==9550== HEAP SUMMARY:  
==9550==    in use at exit: 0 bytes in 0 blocks  
==9550==   total heap usage: 2 allocs, 2 frees, 5,120 bytes allocated  
==9550==  
==9550== All heap blocks were freed -- no leaks are possible  
==9550==  
==9550== For lists of detected and suppressed errors, rerun with: -s  
==9550== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Valgrind Result

Appendix D - Juliet CWE-401 Test Case Logs

TP - Correctly identified leak

FP - Incorrectly identified leak

FN - Missed Leak

TN - Correctly identified no leak

Test Case	Total Functions	TP	FP	FN	TN
CWE401_Memory_Leak_wchar_t_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_int_calloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_07.c	5	1	1	0	3
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_calloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_16.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_21.c	7	1	2	0	4
CWE401_Memory_Leak_char_calloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_char_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_15.c	3	1	2	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_44.c	5	0	0	1	4
CWE401_Memory_Leak_twoIntsStruct_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_12.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_char_15.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_02.c	3	1	2	0	0
CWE401_Memory_Leak_int_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_03.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_wchar_t_calloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_int_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_11.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_calloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_12.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_45.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_calloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_10.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_calloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_11.c	5	1	0	0	4

CWE401_Memory_Leak_malloc_realloc_char_09.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_10.c	3	1	2	0	0
CWE401_Memory_Leak_char_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_char_04.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_07.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_wchar_t_06.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_14.c	3	1	2	0	0
CWE401_Memory_Leak_int_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_05.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_18.c	2	1	1	0	0
CWE401_Memory_Leak_twoIntsStruct_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_12.c	2	1	1	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_18.c	2	1	1	0	0
CWE401_Memory_Leak_twoIntsStruct_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_03.c	3	1	2	0	0
CWE401_Memory_Leak_int_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_09.c	3	1	2	0	0
CWE401_Memory_Leak_int_malloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_char_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_strdup_wchar_t_08.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_17.c	3	1	0	0	2

CWE401_Memory_Leak_wchar_t_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_twoIntsStruct_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_14.c	3	1	2	0	0
CWE401_Memory_Leak_char_malloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_wchar_t_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_11.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int64_t_14.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_wchar_t_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_17.c	2	1	1	0	0
CWE401_Memory_Leak_wchar_t_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_12.c	3	1	0	0	2
CWE401_Memory_Leak_int_calloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_calloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_char_realloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_char_07.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_08.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_char_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_11.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int_06.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_13.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_char_13.c	3	1	2	0	0
CWE401_Memory_Leak_char_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_09.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_03.c	3	1	2	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_12.c	2	1	1	0	0
CWE401_Memory_Leak_char_calloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_10.c	3	1	2	0	0
CWE401_Memory_Leak_int_calloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_44.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_04.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_06.c	3	1	2	0	0
CWE401_Memory_Leak_int_calloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_17.c	3	1	0	0	2
CWE401_Memory_Leak_char_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_12.c	2	1	1	0	0
CWE401_Memory_Leak_int_calloc_10.c	5	1	0	0	4

CWE401_Memory_Leak_malloc_realloc_int_04.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_wchar_t_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_02.c	3	1	2	0	0
CWE401_Memory_Leak_char_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_char_21.c	7	1	0	0	6
CWE401_Memory_Leak_strdup_wchar_t_01.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_05.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_04.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int_13.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_wchar_t_12.c	2	1	1	0	0
CWE401_Memory_Leak_wchar_t_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_realloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_08.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_wchar_t_05.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_calloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_45.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_int_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_10.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_01.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_calloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_07.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_wchar_t_07.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_wchar_t_32.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_char_07.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_02.c	5	1	0	0	4

CWE401_Memory_Leak_char_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_44.c	5	0	0	1	4
CWE401_Memory_Leak_malloc_realloc_int64_t_04.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int64_t_07.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_09.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_char_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_wchar_t_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_char_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_char_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_10.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_13.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_15.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_45.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_strdup_wchar_t_14.c	5	1	2	0	2
CWE401_Memory_Leak_wchar_t_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_08.c	3	1	2	0	0
CWE401_Memory_Leak_int_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_wchar_t_calloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_18.c	2	1	1	0	0
CWE401_Memory_Leak_char_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_41.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_malloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_21.c	7	1	1	0	5
CWE401_Memory_Leak_twoIntsStruct_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_12.c	2	1	1	0	0
CWE401_Memory_Leak_char_calloc_21.c	7	1	1	0	5
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_17.c	2	1	1	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_17.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_12.c	3	1	1	0	1

CWE401_Memory_Leak_int_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_02.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int_11.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_char_16.c	2	1	1	0	0
CWE401_Memory_Leak_wchar_t_calloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_08.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_05.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_wchar_t_18.c	3	1	0	0	2
CWE401_Memory_Leak_int_calloc_44.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_01.c	2	1	1	0	0
CWE401_Memory_Leak_char_realloc_07.c	5	1	1	0	3
CWE401_Memory_Leak_char_realloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_wchar_t_realloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_02.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_01.c	2	1	1	0	0
CWE401_Memory_Leak_wchar_t_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_realloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_char_realloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_14.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_wchar_t_42.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_strdup_char_10.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_03.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_32.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_07.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_char_03.c	3	1	2	0	0
CWE401_Memory_Leak_int_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_44.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_int64_t_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int_malloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_int64_t_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_calloc_41.c	5	1	0	0	4

CWE401_Memory_Leak_int_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_02.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_15.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_wchar_t_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_41.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int_09.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_wchar_t_07.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_int64_t_malloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_05.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_08.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_16.c	2	1	1	0	0
CWE401_Memory_Leak_strdup_wchar_t_31.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_17.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_wchar_t_13.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int_12.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_malloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int64_t_02.c	3	1	2	0	0
CWE401_Memory_Leak_int_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_wchar_t_34.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_char_malloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_13.c	3	1	2	0	0
CWE401_Memory_Leak_strdup_char_12.c	3	1	1	0	1
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_12.c	3	1	0	0	2
CWE401_Memory_Leak_char_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_14.c	5	1	2	0	2
CWE401_Memory_Leak_int_malloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_malloc_realloc_wchar_t_15.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_realloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_11.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_02.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_malloc_05.c	5	1	0	0	4

CWE401_Memory_Leak_struct_twoIntsStruct_calloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int_calloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_malloc_realloc_wchar_t_11.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_10.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_09.c	3	1	2	0	0
CWE401_Memory_Leak_int_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_int_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_01.c	2	1	1	0	0
CWE401_Memory_Leak_wchar_t_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int64_t_06.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_calloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_08.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_18.c	2	1	1	0	0
CWE401_Memory_Leak_char_calloc_07.c	5	1	1	0	3
CWE401_Memory_Leak_char_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_strdup_wchar_t_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_13.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_char_06.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_calloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_twoIntsStruct_calloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int_16.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_realloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int_17.c	2	1	1	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int64_t_18.c	2	1	1	0	0
CWE401_Memory_Leak_strdup_wchar_t_04.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_21.c	7	1	1	0	5
CWE401_Memory_Leak_malloc_realloc_wchar_t_01.c	2	1	1	0	0
CWE401_Memory_Leak_char_realloc_12.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int64_t_16.c	2	1	1	0	0
CWE401_Memory_Leak_int_realloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_wchar_t_calloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_11.c	5	1	2	0	2
CWE401_Memory_Leak_char_calloc_17.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int64_t_13.c	3	1	2	0	0
CWE401_Memory_Leak_char_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_wchar_t_04.c	3	1	2	0	0
CWE401_Memory_Leak_int_calloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_15.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_07.c	5	1	0	0	4

CWE401_Memory_Leak_int_malloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_char_17.c	2	1	1	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_44.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_wchar_t_05.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_wchar_t_06.c	3	1	2	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_04.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_11.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_wchar_t_03.c	3	1	2	0	0
CWE401_Memory_Leak_int_realloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_int_realloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_wchar_t_calloc_21.c	7	1	1	0	5
CWE401_Memory_Leak_twoIntsStruct_realloc_12.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_twoIntsStruct_realloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_malloc_realloc_char_05.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int64_t_17.c	2	1	1	0	0
CWE401_Memory_Leak_char_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_malloc_realloc_wchar_t_09.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_calloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_32.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_realloc_07.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_int_03.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_realloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_11.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_14.c	5	1	2	0	2
CWE401_Memory_Leak_malloc_realloc_wchar_t_18.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_wchar_t_21.c	7	1	1	0	5
CWE401_Memory_Leak_int_realloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_01.c	2	1	1	0	0
CWE401_Memory_Leak_int64_t_malloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_12.c	3	1	1	0	1
CWE401_Memory_Leak_twoIntsStruct_malloc_41.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_char_08.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_realloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_calloc_44.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_char_realloc_14.c	5	1	1	0	3
CWE401_Memory_Leak_twoIntsStruct_malloc_10.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_wchar_t_10.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_char_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_int_calloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_int_calloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_strdup_wchar_t_03.c	5	1	0	0	4

CWE401_Memory_Leak_malloc_realloc_int64_t_15.c	3	1	2	0	0
CWE401_Memory_Leak_wchar_t_realloc_02.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_15.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_char_calloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_malloc_05.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_strdup_char_34.c	3	1	0	0	2
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_int64_t_realloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_int_realloc_42.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_calloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_06.c	3	1	2	0	0
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_11.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_04.c	3	1	2	0	0
CWE401_Memory_Leak_char_malloc_18.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_malloc_09.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_realloc_16.c	3	1	0	0	2
CWE401_Memory_Leak_twoIntsStruct_calloc_01.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int64_t_05.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_21.c	7	1	1	0	5
CWE401_Memory_Leak_struct_twoIntsStruct_realloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_wchar_t_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_int_malloc_08.c	5	1	0	0	4
CWE401_Memory_Leak_char_calloc_06.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_31.c	3	1	0	0	2
CWE401_Memory_Leak_malloc_realloc_int_14.c	3	1	2	0	0
CWE401_Memory_Leak_int64_t_malloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_06.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_45.c	5	1	0	0	4
CWE401_Memory_Leak_int64_t_calloc_13.c	5	1	0	0	4
CWE401_Memory_Leak_struct_twoIntsStruct_malloc_31.c	3	1	0	0	2
CWE401_Memory_Leak_int_malloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_malloc_realloc_struct_twoIntsStruct_16.c	2	1	1	0	0
CWE401_Memory_Leak_malloc_realloc_wchar_t_14.c	3	1	2	0	0
CWE401_Memory_Leak_twoIntsStruct_malloc_21.c	7	1	0	0	6
CWE401_Memory_Leak_struct_twoIntsStruct_calloc_15.c	5	1	0	0	4
CWE401_Memory_Leak_malloc_realloc_twoIntsStruct_09.c	3	1	2	0	0
CWE401_Memory_Leak_malloc_realloc_int64_t_01.c	2	1	1	0	0
CWE401_Memory_Leak_strdup_char_42.c	5	1	0	0	4
CWE401_Memory_Leak_twoIntsStruct_malloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_wchar_t_calloc_03.c	5	1	0	0	4
CWE401_Memory_Leak_strdup_char_18.c	3	1	0	0	2
CWE401_Memory_Leak_int64_t_calloc_11.c	5	1	1	0	3